CHƯƠNG 2. QUẢN LÝ DỰ ÁN PHẦN MỀM

Thất bại của nhiều dự án phần mềm lớn trong những năm 1960 và đầu năm 1970 là một chỉ số về những khó khăn của quản lý các dự án phần mềm như *phần mềm chuyển giao trễ hạn, không tin cậy, chi phí vượt nhiều lần dự toán, sản phẩm thể hiện sự nghèo nàn về chức năng*. Theo số liệu của nhóm Standish[3] cho thấy:

- 16% dự án hoàn thành đúng lịch, trong phạm vi ngân sách, đủ tính năng.
- 31% dự án bị ngừng khi đang phát triển.
- 53% dự án đã hoàn thành, có thể hoạt động nhưng vượt quá ngân sách, quá thời gian dự kiến, và chỉ cung cấp một số tính năng đề ra.

Theo thống kê, năm 1995 các công ty Mỹ đã phải chi 81 tỷ USD cho những dự án bị hủy bỏ, 59 tỷ USD đầu tư thêm cho các dự án không đúng kế hoạch.

Ở Việt Nam, chẳng hạn dự án "*Hệ thống điện tử xử lý thông tin tại SeaGames 22 VN*" dự kiến kinh phí là 15 tỷ VND, nhưng mới đến tháng 06/2003 đã chi tới 90 tỷ VND.

Những dữ liệu trên có thấy rằng, các dự án phần mềm cần được quản lý để tránh những thất bại, mặc dù quản lý tốt chưa đảm bảo dự án thành công, nhưng quản lý tồi luôn dẫn đến sự thất bại của dự án.

Người quản lý dự án phần mềm phải làm các công việc như một người quản lý một dự án bất kỳ. Tuy nhiên, công nghệ phầm mềm có nhiều điểm khác biệt với các loại công nghệ khác, vì thế việc quản lý các dự án phần mềm đặc biệt khó khăn.

- *Phần mềm là sản phẩm không nhìn thấy được*: Người quản lý dự án không thể nhìn thấy, cầm nắm được phần mềm mà chỉ nhận biết nó qua tiến trình dự án diễn ra, những công việc nào đã và đang thực hiện, những tài liệu cần và đã xét duyệt.
- *Không có một tiến trình phần mềm chuẩn*: Ta không thể có được một sự hiểu biết đầy đủ, rõ ràng về mối quan hệ giữa tiến trình phần mềm và sản phẩm. Mặc dù sự hiểu biết của chúng ta về tiến trình phần mềm đã được nâng lên đáng kể trong vài năm gần đây, nhưng ta vẫn không thể dự kiến trước mọi bất định. Mỗi tiến trình cụ thể luôn nảy sinh những vấn đề, những rủi ro riêng.
- *Các dự án phần mềm lớn thường là dự án mới*: Các dự án phần mềm luôn khác với các dự án trước đó. Bởi vậy, người quản lý khó sử dụng kinh nghiệm đã có để làm giảm bất định khi lập kế hoạch. Họ rất khó khăn để có thể dự kiến hết các vấn đề nảy sinh. Hơn nữa, sự thay đổi nhanh chóng trong kỹ thuật máy tính, và truyền thông làm cho nhiều vấn đề vượt ra khỏi tầm kiến thức, và kinh nghiệm tích lũy của người quản lý, khả năng đáp ứng cho các dự án mới là rất hạn chế. Phần lớn các hệ thống phần mềm luôn là mới và đòi hỏi luôn đổi mới kỹ thuật. Vì vậy, nhiều vấn đề trở nên nan giải cho nhà quản lý, đặc biệt trong hoạt động lập kế hoạch và lập lịch.
- *Sự thay đổi là cố hữu đối với các dự án phần mềm*: Sự thay đổi về yêu cầu của khách hàng từ nhu cầu nghiệp vụ, sự thay đổi về công nghệ do sự tiến bộ thường xuyên của công nghệ thông tin, sự thay đổi về môi trường đích mà phần mềm được xây dựng sẽ hoạt động, … Tất cả những yếu tố này luôn tác động lên quá trình phát triển mà người quản lý dự án buộc phải đối mặt.

2.1. Dự án và quản lý dự án

2.1.1. Khái niệm về dự án

Dự án là một tập hợp các công việc được một hoặc một nhóm người thực hiện, nhằm đạt đến một kết quả dự kiến là *một sản phẩm xác định, trong thời gian nhất định, với kinh phí dự kiến*. Như vậy, khái niệm dự án liên quan đến ba yếu tố cụ thể sau.

o Kết quả dự án là *một sản phẩm hoàn toàn xác định*. Vì vậy, sản phẩm cần được mô tả về các đặc trưng, tính năng tác dụng, giá trị sử dụng, hiệu quả mang lại,…

o *Do một nhóm người thực hiện*. Tất nhiên cũng có dự án do một người thực hiện nhưng hữu hạn. Vì có nhiều người thực hiện, nên trước hết *những người thực hiện phải có những kỹ năng và nghiệp vụ phù hợp* đáp ứng yêu cầu tạo ra sản phẩm. Hơn nữa, cần có cách *tổ chức quản lý* làm việc sao cho khai thác tốt tiềm năng và năng lực của mỗi người, các tài nguyên cần thiết để tạo ra sản phẩm một cách hiệu quả.

o *Có những giới hạn* đặt lên các hoạt động dự án, đó là thời gian và kinh phí cần thiết được sử dụng.

Các hoạt động dự án có các đặc trưng sau.

o Sản phẩm đơn chiếc - duy nhất.

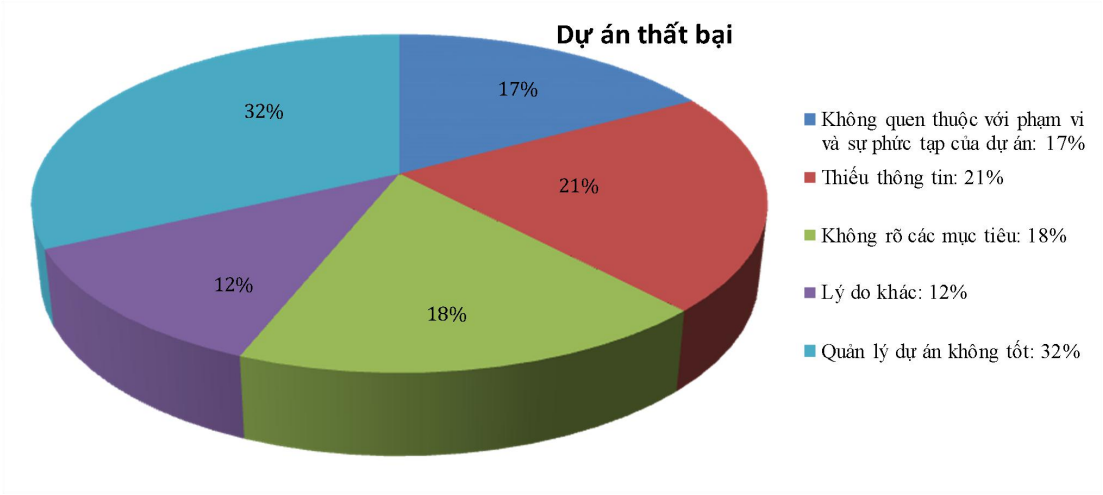
o Chỉ làm một lần.

o Người tham gia thường có chuyên môn khác nhau, được tập hợp lại.

o Có lịch trình chặt chẽ.

o Kết quả không chắc chắn: Sản phẩm đáp ứng yêu cầu và thỏa mãn ràng buộc - dự án thành công, ngược lại - dự án thất bại.

Các đặc trưng này đặt ra nhiều vấn đề cần được nghiên cứu và giải quyết trong hoạt động triển khai dự án. Mặt khác, một dự án thất bại thường một số nguyên nhân sau.



Hình 2. 1. Dự án thất bại

2.1.2. Quản lý dự án

Mục tiêu của quản lý dự án nhằm giúp việc quản lý các tác vụ trở nên dễ dàng hơn bằng cách cung cấp một số ngữ cảnh trong quản lý dự án. Tuy nó không là công thức dẫn đến thành công, nhưng nó hiện diện để dễ dàng quản lý dự án và làm tăng khả năng thành công của dự án phần mềm.

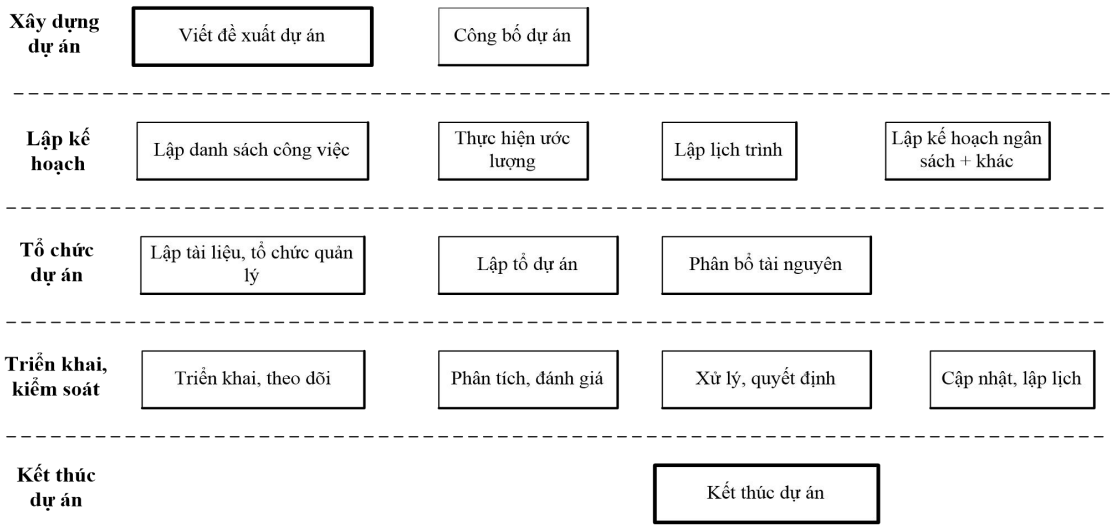
Quản lý dự án là đề ra một sản phẩm cuối cùng, cụ thể.

o Đúng hạn.

o Trong phạm vi ngân sách hay nguồn tài chính cho phép.

o Phù hợp theo các đặc tả.

o Với một mức độ chất lượng để phục vụ các nhu cầu kinh doanh, và đáp ứng các tiêu chuẩn chuyên môn và kỳ vọng của công tác quản lý.



Hình 2. 2. Bức tranh tổng thể tiến trình triển khai một dự án

Cho đến nay, chưa có định nghĩa nào về quản lý dự án là đầy đủ và chính xác. Quản lý dự án được xem là việc vận dụng các kiến thức, phương pháp, công cụ và kỹ thuật để triển khai các hoạt động của dự án nhằm tạo ra sản phẩm theo đúng các yêu cầu được đặt ra. Công việc quản lý dự án trải đều trong suốt vòng đời của dự án, từ lúc hình thành dự án, lập kế hoạch, thực hiện dư án đến kết thúc dự án. Bức tranh tổng quát tiến trình triển khai một dự án minh họa như hình 2.2.

Nhìn một cách bao quát, các nội dung chính của việc triển khai một dự án gồm những hoạt động sau đây.

- *Khởi tạo, xác định dự án*
  - Viết bản đề xuất.
  - Thiết lập quan hệ với khác hàng.
  - Xây dựng dự án sơ bộ.
  - Thiết lập thủ tục quản lý.
  - Thiết lập môi trường quản lý và tài liệu dự án.
- *Lập kế hoạch dự án*
- *Tổ chức thực hiện*
- *Kết thúc dự án*

Viết bản đề xuất. Giai đoạn đầu của một dự án phần mềm là viết đề xuất thực hiện dự án. Bản đề xuất cần mô tả các đối tượng của dự án, cách thức triển khai dự án. Thường thì nội dung phải bao gồm các ước lượng sơ bộ chi phí, và lịch trình thực hiện dự án qua các mốc. Có thể phải trình bày rõ vì sao hợp đồng dự án được trình ra cho một tổ chức hay một đội cụ thể.

Viết bản đề xuất là công việc quan trọng. Nó phụ thuộc vào việc có đủ cơ sở hay không để dự án được chấp nhận, và hợp đồng đưa ra được ký kết. Không có một hướng dẫn cụ thể và đầy đủ nào cho nhiệm vụ này. Công việc này đòi hỏi những kỹ năng thu nhận được kinh nghiệm của người viết.

Những hoạt động khác trong giai đoạn xác định dự án đơn thuần là những hoạt động quản lý có tính nghiệp vụ.

2.2. Lập kế hoạch dự án

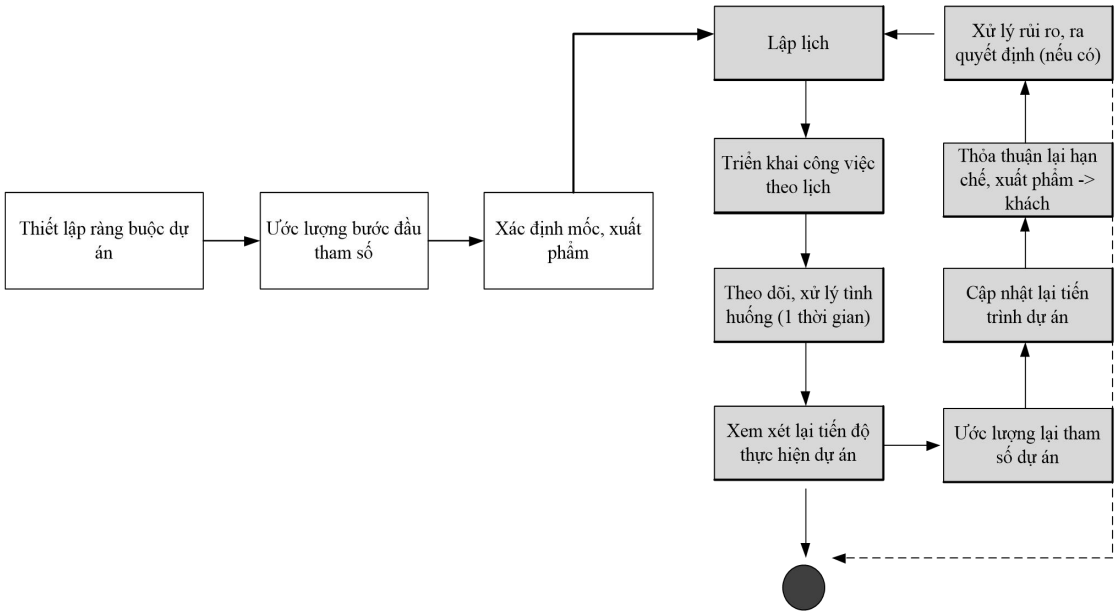
Việc quản lý hiệu quả một dự án phần mềm phụ thuộc trước hết vào việc lập kế hoạch tiến độ của dự án. Người quản lý dự án phải dự trước mọi công việc cần làm,

những vấn đề có thể phát sinh và chuẩn bị những giải pháp thích hợp cho những vấn đề đó. Một kế hoạch nên được sử dụng như một công cụ đắc lực cho quản lý dự án.

Kế hoạch	Mô tả
Kế hoạch về chất lượng	Mô tả các thủ tục và chuẩn về chất lượng được sử dụng trong dự án.
Kế hoạch thẩm định	Mô tả cách thức, nguồn lực và tiến độ sử dụng cho việc thẩm định hệ thống.
Kế hoạch quản lý cấu hình	Mô tả các thủ tục và cấu trúc quản lý cấu hình được sử dụng.
Kế hoạch bảo trì	Dự đoán các yêu cầu bảo trì của hệ thống, chi phí bảo trì và các cố gắng cần thiết.
Kế hoạch phát triển nhân lực	Mô tả cách phát triển các kỹ năng và kinh nghiệm của các thành viên dự án.

Bảng 2. 1. Các kỹ năng thực tiễn

2.2.1. Tiến trình lập kế hoạch quản lý dự án



Hình 2. 3. Tiến trình lập kế hoạch dự án

Biểu đồ trên mô tả tiến trình lập kế hoạch dự án cho việc phát triển phần mềm. Qua biểu đồ, chúng ta thấy được rằng việc lập kế hoạch là một quá trình lặp và chỉ kết thúc khi dự án đó hoàn thành. Mỗi khi đã có cụ thể các thông tin về dự án, cần xem xét lại dự án một cách định kỳ. Mục tiêu tổng thể của doanh nghiệp là nhân tố quan trọng cần được xét đến khi hình thành kế hoạch dự án. Quá trình lập kế hoạch bắt đầu bằng việc đánh giá những ràng buộc (thời gian giao sản phẩm được yêu cầu, số lượng nhân viên, tổng ngân sách, …) ảnh hưởng đến dự án. Nó được vạch ra phù hợp với những tham số ước lượng của dự án như cấu trúc, kích cỡ và việc phân bổ các chức năng của dự án.

Các mốc tiến độ và chuyển giao sản phẩm được xác định. Tiến trình sau đó sẽ đi vòng lặp. Một lịch trình cho dự án được lập ra và các hoạt động được xác định trong lịch được khởi động hoặc được tiếp tục triển khai. Sau khoảng thời gian (2 - 3 tuần),

tiến độ được xem xét và ghi chú các điểm không nhất quán. Do các tham số của dự án đánh giá ban đầu chỉ là dự kiến, vì vậy kế hoạch cần phải được bổ sung và điều chỉnh thường xuyên.

Khi kế hoạch bị trễ, nhà quản lý phải đàm phán lại ràng buộc về thời hạn chuyển giao sản phẩm với khách hàng. Nếu việc đàm phán không thành công và lịch trình không thể đáp ứng thì cần phải tổ chức một buổi xem xét lại về mặt kỹ thuật của dự án. Mục tiêu của buổi họp này là tìm ra những cách tiếp cận khác để phát triển dự án đúng yêu cầu. Các vấn đề gần như chỉ xuất hiện trong quá trình thực hiện dự án. Do đó cần phải tính đến cả những yếu tố bất ngờ để không đàm phán với khách hàng ở các vòng lặp của dự án.

2.2.2. Lập kế hoạch quản lý dự án

Kế hoạch dự án thiết lập những tài nguyên có thể sử dụng cho dự án, phân chia các công việc, và lịch trình thực hiện công việc. Kế hoạch dự án có thể là một tài nguyên duy nhất bao gồm nhiều loại khác nhau như đã giới thiệu ở trên. Trong trường hợp khác, kế hoạch dự án chỉ liên quan đến tiến trình phát triển phần mềm.

Sự thay đổi chi tiết trong kế hoạch dự án phụ thuộc từng kiểu dự án vào từng tổ chức thực hiện. Tuy nhiên, hầu hết các kế hoạch đều bao gồm những phần sau.

- *Giới thiệu*. Phần này mô tả các mục đích của dự án và đưa ra các ràng buộc (như ngân sách, thời gian,…) tác động lên dự án.
- *Tổ chức dự án*. Phần này mô tả cách quản lý, các đội phát triển, xác định vai trò và trách nhiệm của họ trong bộ phận.
- *Phân tích rủi ro*. Phần này mô tả các rủi ro có thể, khả năng xảy ra các rủi ro đó và các chiến lược giảm thiểu rủi ro được đề xuất.
- *Các yêu cầu tài nguyên phần cứng và phần mềm*. Phần này mô tả các phần cứng và phần mềm cần có để thực hiện việc phát triển. Nếu phải mua phần cứng, cần ước lượng chi phí và lịch trình tiếp nhận.
- *Phân chia công việc*. Phần này mô tả việc phân dự án thành những hoạt động cụ thể và xác định các mốc xuất phẩm đi kèm với mỗi hoạt động.
- *Lịch trình thực hiện*. Phần này mô tả các hoạt động cần thực hiện và sự phụ thuộc giữa chúng, đánh giá thời gian cần thiết để đạt tới mỗi mốc và vị trí của từng người trong các hoạt động.
- *Các cơ chế giám sát và báo cáo*. Phần này mô tả các báo cáo quản lý cần đưa ra và cơ chế thực hiện việc báo cáo.

Kế hoạch dự án cần được định và lặp lại trong quá trình thực hiện dự án. Một vài phần như lịch trình dự án sẽ thay đổi thường xuyên, những phần khác tương đối ổn định. Tài liệu phải được tổ chức cho phép dễ dàng sửa đổi và thay thế những phần còn được sử dụng tiếp.

2.3. Lập lịch dự án

2.3.1.Tiến trình lập lịch

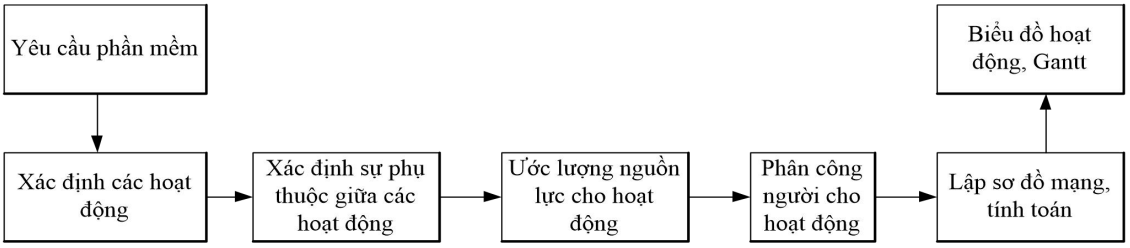
Lập lịch dự án là một công việc đòi hỏi khắc khe và chính xác, một hoạt động đặc trưng của quản lý dự án. Những nhà quản lý phải làm chi tiết các công việc cần thực hiện, ước lượng thời gian và các tài nguyên cần thiết để hoàn thành chúng, tổ chức chúng theo một chuỗi tuần tự liên kết chặt chẽ với nhau theo đúng logic.

Việc xác định lịch trình dự án tốt là việc rất phức tạp do các dự án phần mềm khác nhau có thể sử dụng các mô hình tiến trình (mô hình thác nước, mô hình xoắn ốc,…), các phương pháp phân tích và thiết kế khác nhau (cấu trúc hay đối tượng),

ngôn ngữ thực thi khác nhau. Nếu dự án thiên về kỹ thuật, các đánh giá ban đầu hầu như là khá tốt, ngay cả khi người quản lý cần phải xem xét đến những tình huống có thể xảy ra. Lúc này, lập lịch phần mềm không khác so với việc lập lịch các kiểu dự án lớn khác. Các dự án phần mềm lớn thường bị trễ vì các vấn đề không thể dự kiến hết được. Do đó, lịch trình cần được cập nhật liên tục khi có thêm các thông tin chi tiết thực hiện các hoạt động và tiến độ của dự án.

Tiến trình lập lịch dự án được minh họa bởi hình 2.4 bên dưới bao gồm việc phân chia hoạt động của dự án thành các công việc riêng rẽ, và đánh giá thời gian cần thiết để hoàn thành chúng. Người lập lịch dự án cần phải phối hợp sao cho các hoạt động có thể thực hiện song song, và tổ chức công việc sao có thể sử dụng tối đa các nguồn lực.

Xây dựng bảng công việc chi tiết để hoàn thành dự án là một việc khó khăn. Người quản lý phải đưa ra được tất cả các công việc cần làm để hoàn thành dự án. Bảng công việc được xác định tốt là cơ sở để xác định các bước hoàn thành dự án, làm cơ sở cho việc phân bổ các nguồn lực, cho việc xác định trách nhiệm cá nhân và xây dựng lịch trình thực hiện dự án.



Hình 2. 4. Tiến trình lập lịch dự án

Sau khi đã xác định được công việc cần ước lượng thời gian thực hiện mỗi công việc. Nguyên tắc ước lượng thời gian là tiến hành cho từng công việc nhỏ, từ đó ước lượng cho những công việc gộp hơn. Việc ước lượng thời gian có thể theo phương pháp kinh nghiệm. Trong phương pháp PERT (*Program Evaluation and Review Technicques*), ước lượng thời gian công việc dựa trên ba ước lượng cơ bản sau.

o *Ước lượng khả dĩ* (*ML - Most likely*) là thời gian cần thiết để hoàn thành công việc trong điều kiện "bình thường".

o *Ước lượng lạc quan* (*MO - Most Optimistic*) là thời gian cần thiết để thực hiện công việc trong điều kiện "lý tưởng".

o *Ước lượng bi quan* (*MP - Most Pessimistic*) là thời gian cần thiết để hoàn thành công việc trong điều kiện "tồi nhất".

Ước lượng cuối cùng thời gian thực hiện công việc được tính theo công thức sau.

$$\mathbf{t_{cv}} = (MO + 4.ML + MP)/6$$

Các hoạt động của dự án thường diễn ra ít nhất một tuần. Khoảng thời gian lớn nhất cho một hoạt động khoảng từ tám đến mười tuần. Nếu nó diễn ra thời gian dài hơn thì nên tiếp tục phân chia hoạt động thành những công việc nhỏ hơn.

Người quản lý cũng phải đánh giá các tài nguyên cần thiết để hoàn thành mỗi công việc. Tài nguyên có thể là con người, dung lượng đĩa cần thiết trên máy chủ, thời gian cần thiết cho những phần cứng đặc biệt như bộ giả lập, chi phí đi lại cần thiết cho các nhân viên dự án,… Việc dự toán tài nguyên cho các công việc được tiến hành dựa trên kinh nghiệm hay số liệu thống kê của các dự án đã thực hiện.

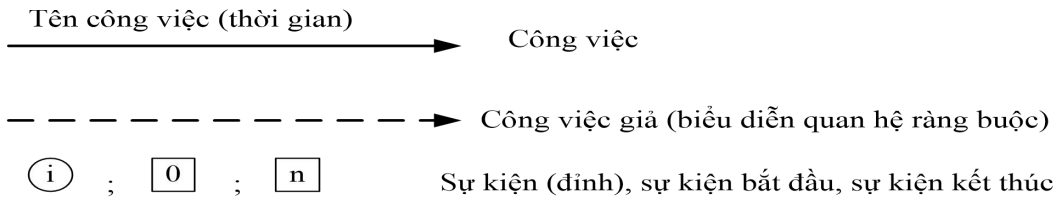
Một quy tắc tốt là, khi đánh giá nếu không có gì sai thì tăng đánh giá lên để bao gồm cả những vấn đề đã thấy trước. Yếu tố bất ngờ bao gồm những vấn đề không thấy

trước cũng có thể được thêm vào sự đánh giá. Yếu tố bất ngờ này phụ thuộc vào kiểu dự án, các tham số tiến trình (thời hạn cuối, các chuẩn,…). Ta nên thêm 30% vào đánh giá ban đầu cho những vấn đề đã biết trước và thêm 20% nữa để bao gồm những vấn đề chưa nghĩ đến.

Lịch dự án thường được đưa ra dưới dạng một tập các đồ thị biểu diễn phân chia công việc. Các công cụ phần mềm thường được sử dụng để trợ giúp công việc quản lý phần mềm (Microsoft Project 2000). Các công cụ trợ giúp việc sinh tự động các biểu đồ, tính toán các tham số, lập báo cáo.

2.3.2. Phương pháp và công cụ lập lịch

Phương pháp đường giăng (*CMP - Critical Path Method*) thường được sử dụng trong hoạt động lập lịch. Phương pháp này sử dụng một sơ đồ mạng các công việc, nó là một đồ thị có hướng, mỗi cung biểu diễn một công việc, mỗi đỉnh biểu diễn một sự kiện là điểm bắt đầu hay kết thúc của một số các công việc. Có một đỉnh *bắt đầu* cho dự án là đỉnh mà tại đó các công việc đầu tiên của dự án đi ra, và một đỉnh *kết thúc*tức là các công việc cuối cùng của dự án đi vào. Trên mỗi cung - công việc có ghi tên công việc và thời gian thực hiện nó và được minh họa như hình sau.



Hình 2. 5. Các ký pháp của sơ đồ mạng

Công Việc	Thời Gian t_{cv}	Đi Sau Công Việc	Công Việc	Thời Gian t_{cv}	Đi Sau Công Việc
a	1	-	k	2	g, i
b	7	-	m	3	i
c	8	-	l	3	i
d	4	-	n	2	k
e	4	a	o	1	l, n
f	3	b	p	2	g, i, h
g	3	c	q	3	g, i, h
h	4	d	r	2	o, p
i	2	e, f	s	1	r, q

Bảng 2. 2. Bảng liệt kê các công việc của dự án

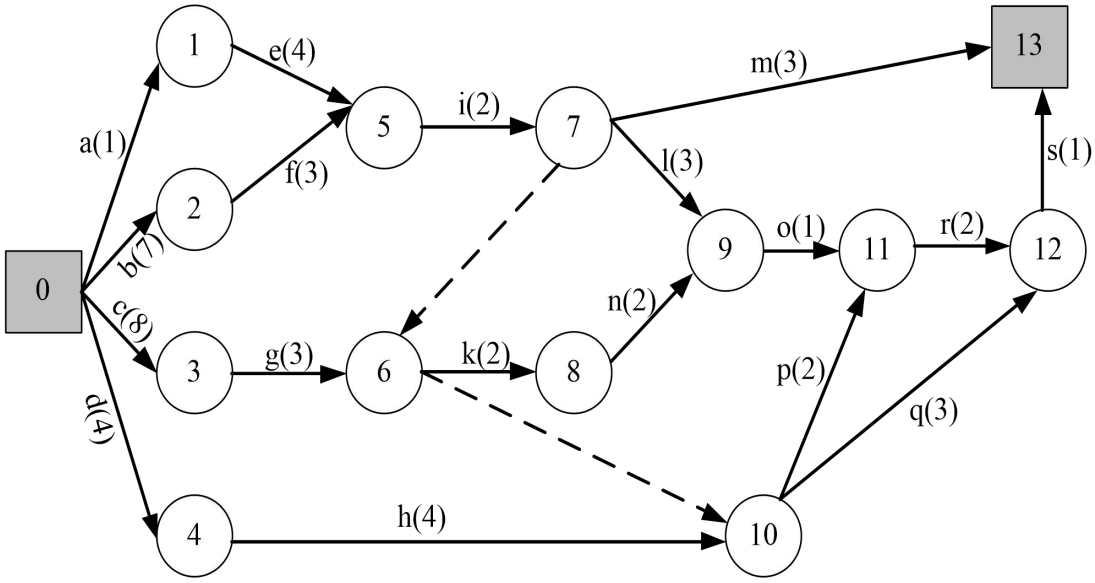
Từ bảng liệt kê các công việc của dự án trên, ta vẽ được sơ đồ mạng các công việc hình 2.6. Dựa trên cơ sở sơ đồ mạng, ta tính được các tham số thời gian cho mỗi đỉnh và mỗi công việc hình 2.6, cụ thể.

o *Thời gian sớm nhất* của mỗi đỉnh được tính từ đỉnh bắt đầu (tính theo chiều xuôi):

- Đỉnh bắt đầu: $\mathbf{t_s(0) = 0}$.
- Đỉnh i (i = 1, 2, 3,…): $\mathbf{t_s(đỉnh\ i)}$ = **Max**{$\mathbf{t_s}$(đỉnh đầu cv) + $\mathbf{t_{cv}}$}, đối với mọi **cv** đi vào đỉnh i.

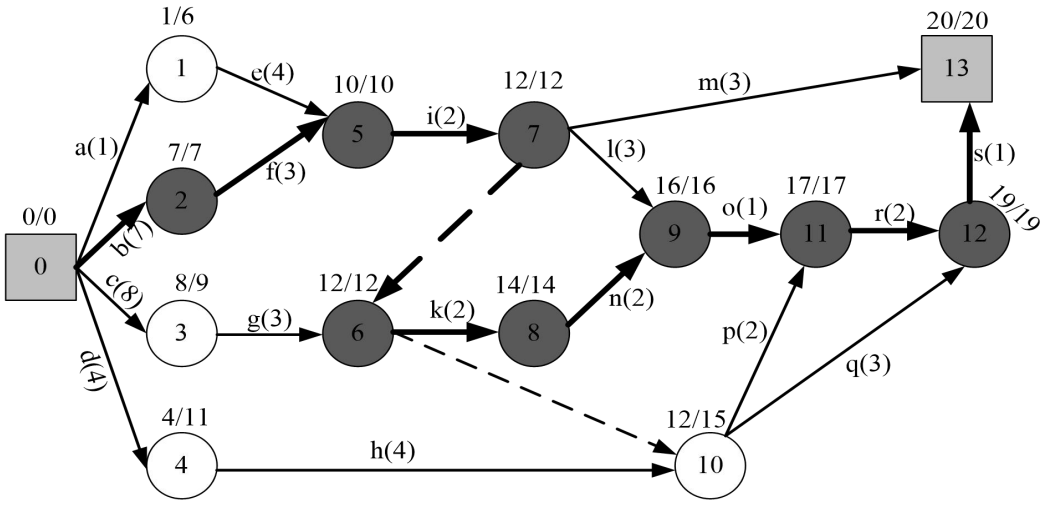
o *Thời gian muộn nhất* của một đỉnh được tính từ đỉnh kết thúc (tính theo chiều ngược).

- Đỉnh kết thúc (n): $t_m(n) = t_s(n)$.
- Đỉnh i (i = n-1, …, 1): $\mathbf{t_m(đỉnh\ i)}$ = **Min**{$\mathbf{t_m}$(đỉnh cuối cv) - $\mathbf{t_{cv}}$}, đối với mọi **cv** đi ra khỏi đỉnh i.



Hình 2. 6. Sơ đồ mạng được vẽ từ bảng dữ liệu cho bởi bảng công việc

Trong đó, mỗi cạnh là một công việc, chữ là tên công việc của nó và số trong ngoặc là thời gian thực hiện.



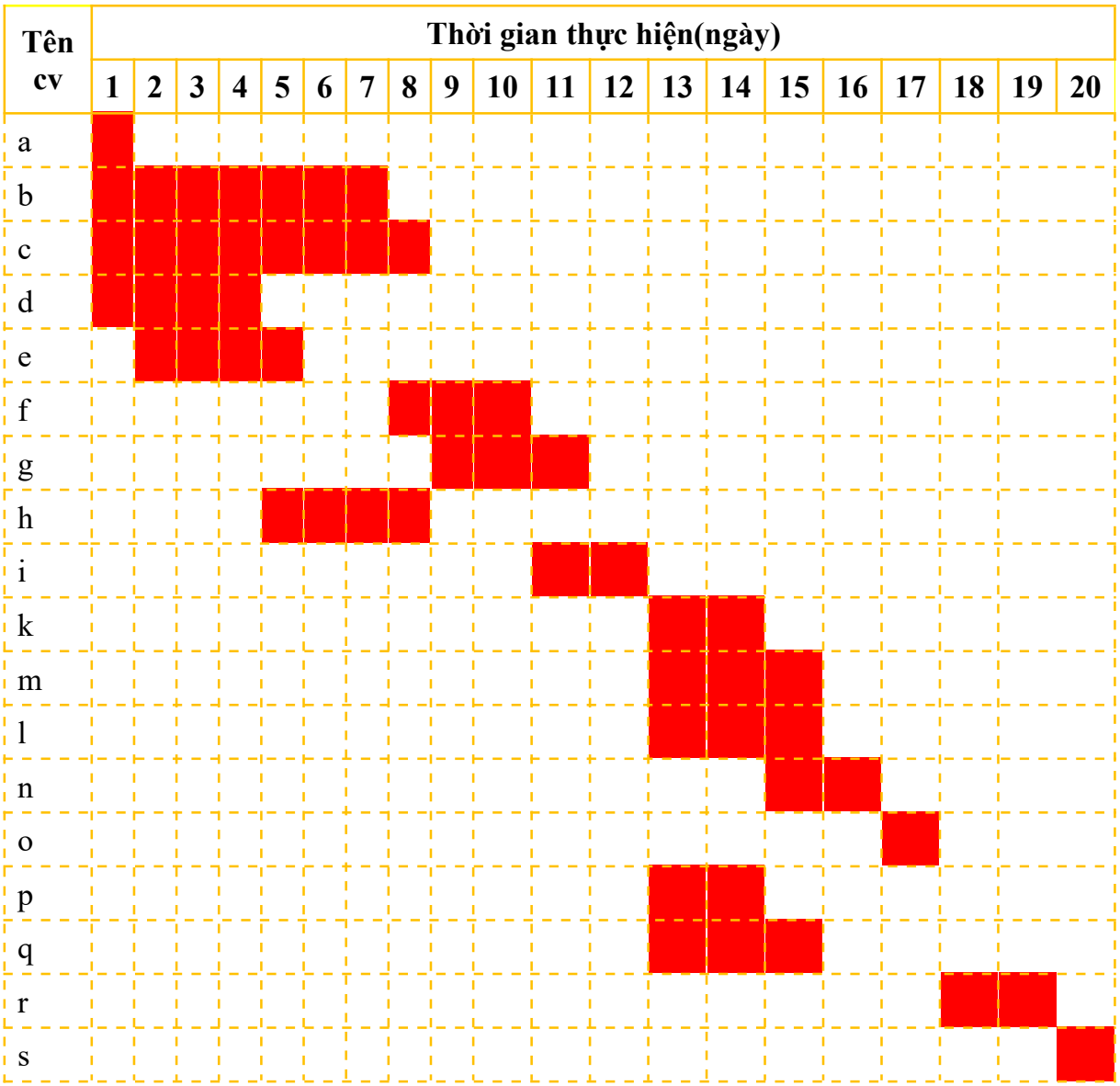
Hình 2. 7. Sơ đồ mạng với các tham số thời gian và đường giăng

Trong đó, 2 số trên mỗi đỉnh là thời gian sớm nhất và thời gian muộn nhất của đỉnh.

- Thời gian dự phòng của mỗi công việc:

$$\mathbf{t_{df}}(cv) = \mathbf{t_m}(\text{đỉnh cuối cv}) - \mathbf{t_s}(\text{đỉnh đầu cv}) - \mathbf{t_{cv}}$$

Những *công việc giăng* (nét đậm trên hình 2.7) là những công việc có thời gian dự phòng bằng 0 ($t_{df}(cv) = 0$). Khi đó *đường giăng* là đường gồm các công việc giăng đi từ đỉnh bắt đầu đến đỉnh kết thúc. Độ dài của đường này là 20 ngày như hình 2.7, và chính là độ dài ngắn nhất cần thiết để thực hiện dự án (vì chưa kể đến các điều kiện ràng buộc khác). Nếu kéo dài thời gian thực hiện của bất kỳ một công việc nào trên đường giăng đều dẫn đến việc kéo dài thời gian thực hiện dự án. Vì thế các công việc giăng được đặc biệt chú ý khi theo dõi và giám sát dự án. Trong trường hợp cần thiết có thể chia nhỏ hơn những công việc giăng (nếu có thể), vẽ lại mạng công việc và tính toán lại, hy vọng rằng có thể rút ngắn thời gian thực hiện dự án.



Bảng 2. 3. Biểu đồ Gantt lịch trình dự án

Sau khi đã tính toán các tham số của công việc và các đỉnh của mạng công việc, ta có thể chuyển nó sang biểu đồ Gantt như bảng 2.3 để dễ quan sát và theo dõi trong quản lý dự án. Trong biểu đồ Gantt, mỗi công việc biểu diễn bằng một khối chữ nhật

có độ dài bằng thời gian bắt đầu sớm nhất của đỉnh mà công việc đó đi ra. Nhìn trên biểu đồ ta có thể thấy rõ ngày kết thúc của mỗi công việc.

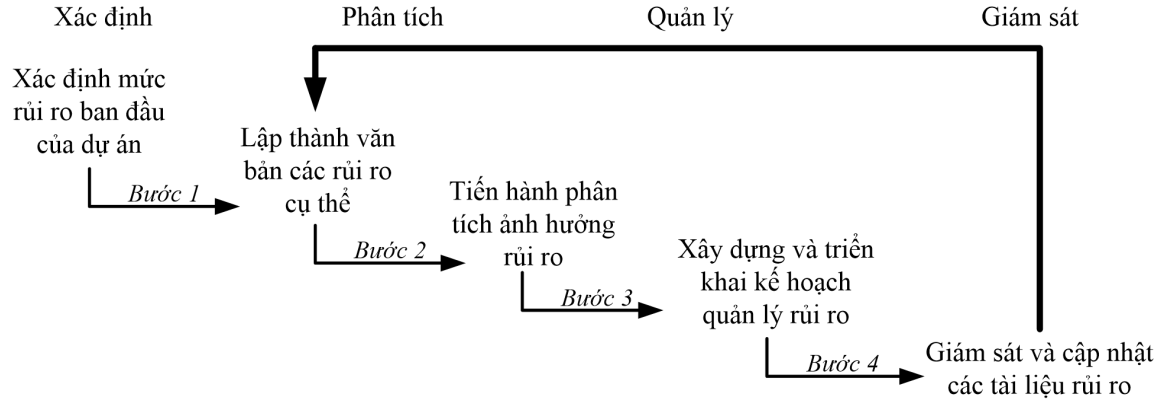
2.4. Quản lý rủi ro

Một nhiệm vụ quan trọng của người quản lý dự án là nhận biết các rủi ro có thể ảnh hưởng đến lịch trình của dự án, hoặc chất lượng của phần mềm đang được phát triển, người quản lý sẽ thực hiện các hoạt động nhằm tránh hay giảm thiểu thiệt hại do các rủi ro này gây ra. Các kết quả phân tích rủi ro được làm thành tài liệu trong kế hoạch dự án cùng với phân tích về hậu quả nếu rủi ro xảy ra. Việc xác định các rủi ro và lập kế hoạch để kiểm soát ảnh hưởng của chúng lên dự án được gọi là *quản lý rủi ro*.

Như vậy, ta có thể định nghĩa rủi ro nhìn nhận dưới các điểm sau.

- Những sự kiện có thể làm phá vỡ một dự án.
- Những điều không chắc chắn, những khoản nợ hay những điểm yếu có thể làm cho dự án không đi theo đúng kế hoạch đã định.

Có thể nói, quy trình quản lý rủi ro nhằm giảm tối thiểu ảnh hưởng của những sự cố không lường trước cho dự án bằng cách xác định và đưa ra những giải pháp tình huống trước khi có những hậu quả xấu xảy ra.



Hình 2. 8. Quy trình quản lý rủi ro

Các rủi ro có thể được phân định thành những loại như sau.

- *Rủi ro dự án* là các rủi ro ảnh hưởng đến lịch trình hoặc tài nguyên dự án.
- *Rủi ro sản phẩm* là các rủi ro ảnh hưởng đến chất lượng hoặc hiệu suất của phần mềm đang được phát triển.
- *Rủi ro nghiệp vụ* là các rủi ro ảnh hưởng đến tổ chức phát triển hoặc thuê phần mềm. Rủi ro nghiệp vụ là ở những rủi ro tiềm ẩn bên trong bởi vì chúng có thể làm sáng tỏ kết quả ngay cả dự án phần mềm tốt nhất. Ứng cử viên cho năm loại rủi ro nghiệp vụ.
  - Xây dựng một sản phẩm tuyệt vời mà chẳng có nhà đầu tư.
  - Xây dựng một sản phẩm chẳng thích hợp với chiến lược sản phẩm tổng thể đáp ứng yêu cầu công ty.
  - Xây dựng một sản phẩm mà lực lượng bán hàng không hiểu làm sao tư vấn.
  - Mất sự hỗ trợ của cấp quản lý cao do thay đổi mối quan tâm hay do thay đổi con người.
  - Mất cam kết về tài chính hay ngân sách.

Tất nhiên, không thể có một sự phân loại hoàn toàn loại trừ nhau. Nếu một lập trình viên có kinh nghiệm rời khỏi dự án, đây có thể là rủi ro dự án (vì việc triển khai hệ thống không có kinh nghiệm có thể bị trễ), và cũng là rủi ro sản phẩm (vì người thay thế có thể không có kinh nghiệm và có thể gây lỗi), và cũng là rủi ro nghiệp vụ (vì kinh nghiệm là thứ không thể mua được cho các công việc trong tương lai). Các rủi ro có thể tác động lên một dự án tùy thuộc vào dự án và môi trường mà phần mềm được phát triển. Tuy vậy, các rủi ro là đa dạng và nhiều khả năng có thể xảy ra. Một số rủi ro đơn thuần là không thể tiên đoán được trước.

Quản lý rủi ro rất quan trọng đối với các dự án phần mềm vì tình trạng không rõ ràng cố hữu mà hầu hết các dự án phải đối mặt. Những điều này xuất phát từ các yêu cầu không được xác định chặt chẽ, các khó khăn trong việc đánh giá thời gian và tài nguyên cần thiết để phát triển phần mềm, sự phụ thuộc rất lớn vào các kỹ năng cá nhân, và sự thay đổi trong các yêu cầu do nhu cầu của khách hàng. Người quản lý dự án cần thấy trước các rủi ro và tầm ảnh hưởng của các rủi ro này lên dự án, sản phẩm và kinh doanh để có những kế hoạch, bước đi xử lý thích hợp.

Rủi ro	Loại rủi ro	Mô tả
Thay đổi nhân viên	Dự án	Nhân viên có kinh nghiệm rời khỏi dự án trước khi nó hoàn tất.
Thay đổi quản lý	Dự án	Có thay đổi về sự ưu tiên trong quản lý của tổ chức.
Phần cứng không sẵn sàng	Dự án	Phần cứng cần thiết cho dự án không được tiếp nhận đúng lịch.
Thay đổi yêu cầu	Dự án và sản phẩm	Số lượng thay đổi trong các yêu cầu lớn hơn dự đoán.
Trễ đặc tả	Dự án và sản phẩm	Các đặc tả các giao diện cần thiết không có sẵn theo đúng lịch.
Đánh giá thấp kích thước	Dự án và sản phẩm	Kích cỡ của hệ thống đã ước lượng quá thấp.
Công cụ CASE có hiệu suất kém	Sản phẩm	Các công cụ CASE hỗ trợ dự án không hoạt động như mong đợi.
Thay đổi mong đợi	Kinh doanh	Công nghệ mà hệ thống đang sử dụng bị thay thế bởi công nghệ mới hơn.
Cạnh tranh sản phẩm	Kinh doanh	Một sản phẩm cạnh tranh được đưa ra thị trường trước khi hệ thống hoàn tất.

Bảng 2. 4. Các rủi ro phần mềm

Tiến trình quản lý rủi ro bao gồm một số giai đoạn sau.

- *Nhận dạng rủi ro*: Xác định công việc dự án, sản phẩm và phát hiện các rủi ro kinh doanh có thể.
- *Phân tích rủi ro*: Đánh giá khả năng xảy ra và hậu quả các rủi ro.
- *Lập kế hoạch rủi ro*: Lập kế hoạch định vị rủi ro bằng cách tránh nó hoặc giảm thiểu ảnh hưởng của nó đến hệ thống.

o *Giám sát rủi ro*: Rủi ro được đánh giá định kỳ và kế hoạch cho việc giảm thiểu rủi ro được xem xét lại mỗi khi có các thông tin cụ thể về rủi ro.

Tiến trình quản lý rủi ro giống như tất cả các kế hoạch dự án khác, là một tiến trình lặp diễn ra trong suốt quá trình thực hiện dự án. Mỗi khi một tập hợp khởi tạo các kế hoạch được đưa ra, tình huống sẽ được giám sát. Khi thông tin về các rủi ro trở nên rõ ràng, chúng có thể được phân tích và phân loại lại, mức độ mới của rủi ro được thiết lập, các giải pháp tránh rủi ro và các kế hoạch cho các trường hợp bất ngờ có thể được thay đổi.

Tiến trình phân tích rủi ro chúng ta đã thiết lập một tập các bộ ba dạng [r_i, l_i, x_i]. Trong đó, r_i là rủi ro thứ i, l_i là sự có thể/xác suất xuất hiện của rủi ro thứ i, và x_i tác động của rủi ro thứ i. Chúng ta còn xem xét cả độ chính xác của ước lượng rủi ro, hậu quả nếu nó xuất hiện, nổ lực để sắp thứ tự ưu tiên các rủi ro đã được phát hiện, bắt đầu bàn về cách kiểm soát/ngăn cản sự có thể xuất hiện.

Theo tham khảo một số nguồn tài liệu, người ta đề nghị các bước quản lý rủi ro cụ thể.

o Họp nhóm nhân viên hiện tại để xác định các nguyên nhân luân chuyển (như điều kiện làm việc kém, lương thấp, thị trường công việc cạnh tranh).

o Một khi dự án bắt đầu phải giả sử việc luân chuyển sẽ xuất hiện, và xây dựng các kỹ thuật để đảm bảo sự liên tục khi thay đổi nhân sự.

o Tổ chức nhóm dự án sao cho thông tin về từng hoạt động được phổ biến rộng rãi.

o Xác định các chuẩn tài liệu và thiết lập cơ chế để đảm bảo rằng tài liệu được xây dựng theo đúng hạn.

o Tiến hành duyệt xét ngang cấp cho toàn bộ công việc nhằm tạo điều kiện mọi người được thúc đẩy tăng tiến độ.

o Xác định các nhân viên dự phòng cho mọi nhà kỹ thuật chủ chốt.

2.5. Giám sát dự án

Đây là công việc được làm hằng ngày, diễn ra xuyên suốt và là công việc của người quản lý dự án (Project Manager) cụ thể, phân phát các yêu cầu được thay đổi, kiểm tra các rủi ro, mục tiêu, độ đo và chất lượng. Báo cáo tình trạng của dự án. Phân phát các vấn đề phát sinh.

Giám sát dự án gồm các giai đoạn cụ thể.

o Lập lịch biểu và phân bố công việc

Mục đích chính của giai đoạn này nhằm điều tiết các thay đổi được tán thành để thực hiện và gia công mà các thay đổi đó phát sinh trong khi thực hiện.

Các bước thực hiện giai đoạn này được liệt kê cụ thể.

- Định vị các yêu cầu thay đổi vào Iteration.
- Phân bổ trách nhiệm. Mô tả công việc và kết quả mong đợi.
- Dự thảo kết quả và các tài nguyên khác.
- Thiết lập lịch biểu.
- Phân thứ tự công việc.

o Giám sát tình trạng dự án

Mục đích chính của giai đoạn này nhằm theo dõi tình trạng hiện tại của dự án. Xác định các tình huống chống lại kế hoạch.

Các bước thực hiện của giai đoạn này cụ thể sau.

- Giữ trạng thái công việc tiến triển theo kế hoạch.
- Ghi nhận các chỉ báo của tiến trình.
- Ghi nhận các chỉ báo của chất lượng.
- Đánh giá các chỉ định và lên kế hoạch đối với các chỉ định này.

o Giải quyết các ngoại lệ và các vấn đề

Mục đích chính của giai đoạn này nhằm giải quyết các vấn đề và ngoại lệ phát sinh trong dự án.

Các bước thực hiện như sau.

- Đánh giá các ngoại lệ và các vấn đề.
- Xác định các hoạt động điều chỉnh kịp thời.
- Ghi nhận các yêu cầu thay đổi và/hoặc thứ tự công việc.

o Báo cáo tình hình dự án

Mục đích chính của giai đoạn này nhằm cung cấp tình trạng thực hiện của dự án để kiểm tra, giải quyết.

o Kiểm tra tiến độ thực hiện dự án

Mục đích của giai đoạn này nhằm kiểm tra tiến độ thực hiện dự án.

Các bước thực hiện kiểm tra như sau.

- Lập lịch biểu cuộc họp đánh giá dự án của những người có thẩm quyền đối với dự án.
- Phát tài liệu trong cuộc họp.
- Chỉ đạo cuộc họp.
- Ghi nhận thời gian.

2.6. Các tiến trình kiểm soát thực hiện dự án

Mục đích của tiến trình kiểm soát nhằm xác định các thông tin, và các tiến trình sẽ được sử dụng để kiểm tra và điều khiển sự tiến triển, chất lượng và các rủi ro.

Quy trình thực hiện gồm các bước sau.

o Xác định các chỉ tiêu trong dự án

Chỉ tiêu (indicators) dự án là những mảng thông tin về dự án, như là một bức tranh về sự tiến triển của dự án dựa vào kế hoạch để phát triển phần mềm. Tiêu biểu là quản lý dự án sẽ quan tâm đến những chỉ tiêu liên quan đến những phạm vi của dự án về công việc, ngân quỹ, chất lượng và các rủi ro. Đối với sự tiến triển của dự án, quản lý dự án sẽ kiểm tra các chỉ số và đưa ra các hành động đúng đắn. Những chỉ tiêu của dự án.

- Tổng chỉ tiêu so với ngân quỹ.
- Kiểm tra mục tiêu (việc đã làm được/ ước lượng thời gian hoàn thành) so sánh với kế hoạch.
- Mật độ lỗi so với các mục tiêu về chất lượng.
- Các chỉ số về rủi ro.
- Xác định các chỉ tiêu về ngân quỹ của dự án, các mục tiêu về chất lượng, bảng biểu và nắm bắt được kế hoạch đo lường, kế hoạch quản lý rủi ro.

o Xác định quy trình báo cáo làm việc nhóm

Trước tiên, chúng ta nên kiểm tra các hàm và các báo cáo của các đội thành viên trong dự án một cách thường xuyên để báo cáo tình trạng của họ. Các hàm này mô tả tiến trình đã định trước dựa vào các gói công việc, các báo cáo mức độ hoàn thành của các tác vụ, sự hoàn thành của các mốc công việc và báo cáo kết quả. Để đảm bảo sự nhất quán về thông tin, có một mẫu chuẩn cho bảng chấm công và báo cáo tình trạng của các đội.

- Xác định quy trình và những giới hạn đối với thao tác hiệu chỉnh.

Để duy trì được kết quả của dự án, quản lý dự án phải xác định các giới hạn về các giá trị/các điều kiện cho mỗi chỉ tiêu dự án. Các điều kiện bị giới hạn được ghi lại trong phần giám sát và kiểm soát dự án (Project Monitoring and Control).

Khi các giới hạn bị vượt quá dự kiến ban đầu, thì nhà quản lý dự án phải điều chỉnh các hành động. Vì sự phụ thuộc và tính nghiêm khắc của dự án, quản lý dự án chỉ có thể giải quyết được các tình huống thuộc quyền của mình. Nếu một tình huống xảy ra ngoài quyền hạng của người quản lý phần mềm, thì người quản lý cần phải đưa ra một bản thay đổi yêu cầu (Change Request) và kích hoạt tiến trình điều khiển thay đổi dự án.

- Xác định tổ chức thực hiện dự án

Xác định cấu trúc tổ chức dự án trong các giới hạn về vị trí, các đội, các trách nhiệm và thứ bậc.

Việc chọn cấu trúc của dự án phụ thuộc vào các đặc điểm của dự án, các ràng buộc bên ngoài cũng như chính sách về tổ chức đã tồn tại…

- Xác định các yêu cầu phân bổ nhân viên

Xác định số lượng, các kỹ năng, kinh nghiệm và phẩm chất của nhân viên được yêu cầu cho dự án.

Quản lý dự án cần ước lượng được số lượng và các kỹ năng của nhân viên hiện có nhằm nâng cao trình độ của các nhân viên, và hiệu quả thực hiện dự án khi tăng nhân viên.

Nhằm hỗ trợ xác định các quy trình quản lý và giải quyết các vấn đề dự án, bao gồm thực hiện các bước sau.

- Xác định các quy trình giải quyết vấn đề

Đầu tiên, xác định các thủ tục để trình bày mỗi loại vấn đề. Những vấn đề cần xem xét.

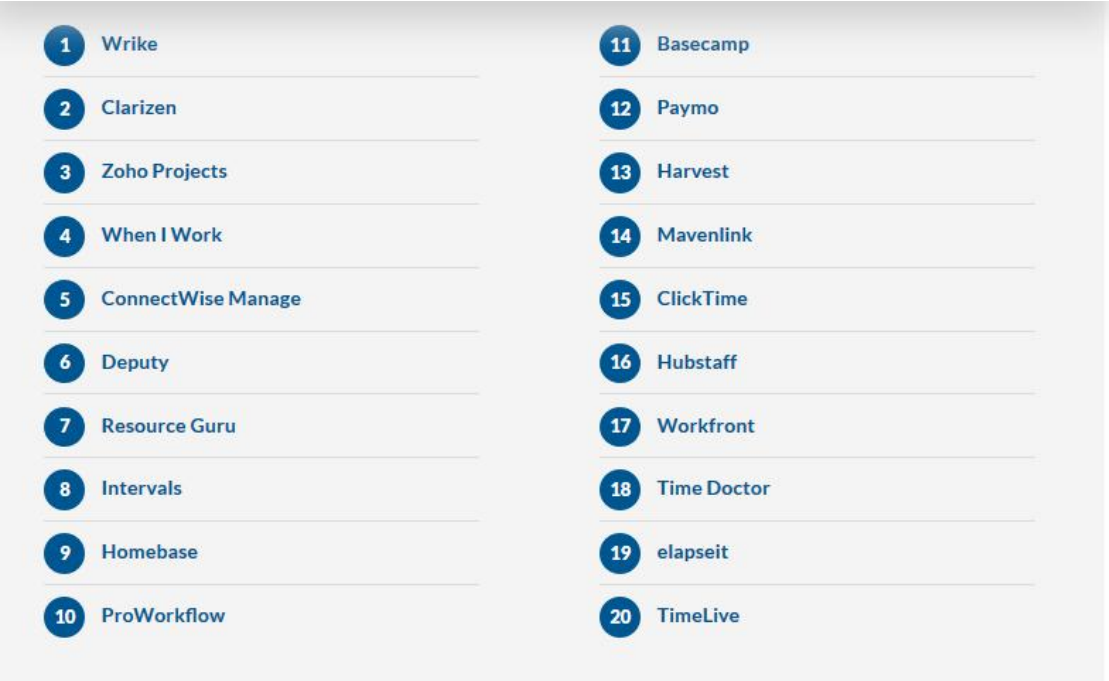
- Phương thức các nhóm sử dụng để đặt vấn đề.
- Ai đánh giá vấn đề và đưa ra các hướng giải quyết tốt nhất?
- Cơ chế gì cho việc cài đặt, lựa chọn các giải pháp?
- Các hành động sẽ được kiểm tra như thế nào là hoàn thành?



Hình 2. 9. Quy trình giải quyết vấn đề

- Công cụ theo dõi tiến độ thực hiện dự án

Hiện nay, để quản lý việc thực hiện và theo dõi tiến độ làm việc của các nhóm thực hiện từ xa và phân tán. Có thể nói, nhà quản lý sử dụng một số phần mềm hàng đầu được minh họa như hình bên dưới.



Hình 2. 10. Công cụ theo dõi tiến độ thực hiện dự án theo thời gian thực

2.7. Kết thúc dự án

Mục tiêu kết thúc dự án là hoàn tất dự án. Kết thúc dự án xảy ra sau pha triển khai thực hiện. Pha bảo trì là đặc trưng cho việc tiếp tục các dự án khác mà mỗi dự án đó được quản lý riêng. Kết thúc dự án bao gồm thực hiện các công việc sau.

- *Đóng dự án lại*: Để đóng lại dự án cần thực hiện một số hoạt động như đánh giá các thành viên và kiến nghị các lợi ích đối với họ, hoàn thiện các tài liệu và các chứng từ thanh toán. Cảm ơn những cộng tác viên đã đóng góp tham gia và hỗ trợ trong quá trình thực hiện công việc.
- *Tổng kết sau dự án*: Mục tiêu là xác định được ưu nhược điểm của sản phẩm, của quá trình hình thành lên nó và quá trình quản lý dự án, từ đó rút ra những kinh nghiệm cần thiết cho các dự án sau.

- *Kết thúc mọi hợp đồng với khách hàng*: Ký kết các bản thanh lý hợp đồng với khách hàng.

Mục đích của kết thúc dự án nhằm đánh giá dự án có được chấp nhận thực hiện hay không. Tức là, chúng ta chuyển giao sản phẩm để khách hàng kiểm tra một cách cẩn thận và chấp nhận sự phân phối của dự án. Cụ thể,

- Lập biểu đồ để đánh giá có chấp nhận thực hiện dự án hay không?

Những thành viên tham gia vào nghiệm thu dự án nên bao gồm đại diện phía khách hàng và đội ngũ quản lý (Project Manager, cùng với những người đứng đầu các nhóm ở các chức năng khác nhau).

Trước tiên xác định các thành viên của cuộc họp, thiết lập ngày, thời gian và nơi tổ chức. Chúng ta phải cung cấp đủ thời gian để những người tham gia kiểm tra dữ liệu của dự án đưa ra các quyết định.

- Phân phát tài liệu trong cuộc họp

Người tổ chức cuộc họp, phân phối các dữ liệu để kiểm tra. Bảo đảm rằng các dữ liệu đó phải đầy đủ và phải có đủ thời gian để các thành viên kiểm tra xem xét chúng. Tối thiểu cần phải có các sưu liệu sau.

- Đánh giá Iteration, bao gồm kiểm tra các thỏa thuận và các kiểm tra lại.
- Sưu liệu kế hoạch phát triển phần mềm - Software Development Plan.

- Chỉ đạo cuộc họp

Trong suốt cuộc họp, các thành viên kiểm tra lại kết quả về các thỏa thuận của sản phẩm mà được báo cáo trong đánh giá lặp thực hiện dự án. Sử dụng các tiêu chuẩn được thỏa thuận từ kế hoạch chấp nhận sản xuất (Product Acceptance Plan), xác định theo các nhóm sau.

- Kiểm toán các kết quả vật chất mà khách hàng nhận được từ các bản phân phối?
- Kiểm toán các kết quả về chức năng, các kết quả việc kiểm tra các thỏa thuận của sản phẩm có chứng minh được sản phẩm đã thỏa mãn các yêu cầu hay chưa?
- Các yêu cầu về huấn luyện khách hàng đã hoàn tất chưa?
- Nếu được yêu cầu, việc cài đặt đã hoàn tất chưa?
- Cuối buổi họp, những người kiểm tra nên đưa ra các quyết định. Nếu một vài tiêu chuẩn thỏa thuận không được thỏa mãn, thì khách hàng sẽ đưa ra quyết định chấp nhận sản phẩm theo một vài điều kiện cần sửa chữa. Trong trường hợp này chúng ta nên tạo ra một Iteration mới giải quyết vấn đề này sinh.

Kết quả của nghiệm thu dự án có thể là một trong các trường hợp sau.

Chấp thuận dự án	Các đại diện phía khách hàng đồng ý với các bản phân phối đã thỏa mãn được các tiêu chuẩn thỏa thuận, và khách hàng sở hữu bản phân phối và dữ liệu hỗ trợ.
Điều kiện thỏa thuận	Đại diện phía khách hàng đồng ý chấp nhận kết quả của dự án, đưa ra các hành động sửa chữa phải hoàn tất.
Không được chấp nhận	Dự án thất bại không đáp ứng được các tiêu chuẩn thỏa mãn, các yêu cầu thêm vào và một dự án thỏa thuận được thực hiện.

Bảng 2. 5. Đánh giá nghiệm thu dự án

Nếu dự án không được chấp nhận bởi khách hàng, thì đội thực hiện dự án nên lập bảng xác định các hành động cần sửa chữa, và đồng ý duyệt lại những đánh giá lặp.

Trong trường hợp chấp nhận các điều kiện (conditional Acceptance), việc đánh giá này chỉ cần xác nhận lại các hành động cần được sửa chữa. Tuy nhiên, nếu dự án đã không chấp nhận (not accepted) thì toàn bộ dự án cần được kiểm tra lại, việc kiểm thử (test) được thi hành.

- Ghi nhận các quyết định trong cuộc họp

Cuối cuộc họp, hồ sơ đánh giá được hoàn tất để lưu lại các thông tin quan trọng về các cuộc tranh luận, và ghi lại kết quả cuộc họp. Nếu kết quả không chấp nhận thì một cuộc họp đánh giá dự án nên được thực hiện sau này.

2.8. Câu hỏi và bài tập

Câu 1. Vì sao phải coi trọng việc quản lý dự án phần mềm? Quản lý dự án phần mềm khác với quản lý dự án khác như thế nào?
Câu 2. Quản lý dự án phần mềm gồm những nội dung gì? Giải thích tóm tắt?
Câu 3. Vẽ sơ đồ mô hình tiến trình quản lý dự án phần mềm?
Câu 4. Lập kế hoạch dự án gồm những nội dung gì?
Câu 5. Trình bày tiến trình lập lịch dự án?
Câu 6. Quản lý rủi ro là gì? Tại sao phải quản lý rủi ro? Mục tiêu quản lý rủi ro?
Câu 7. Trình bày phương pháp đường giăng để lập lịch dự án?
Câu 8. Có những biểu đồ nào được sử dụng để giám sát dự án?
Câu 9. Quy trình thực hiện kiểm soát dự án là gì?

CHƯƠNG 3. XÁC ĐỊNH VÀ ĐẶC TẢ YÊU CẦU PHẦN MỀM

3.1. Tổng quan về yêu cầu phần mềm

3.1.1. Khái niệm yêu cầu phần mềm

Hiểu được bản chất của vấn đề cần phải giải quyết thường là công việc vô cùng phức tạp, đặc biệt khi nó là một hệ thống mới hay một hệ thống đang tồn tai được dùng làm mô hình cho phần mềm muốn tạo ra. Trên thực tế, người ta dễ nhầm lẫn giữa nhu cầu và yêu cầu của người dùng, mà thực sự chúng không phải là như nhau. Một tổ chức có thể quyết định rằng họ "*cần một hệ thống phần mềm trợ giúp việc quản lý nhân lực"*. Nhưng việc đưa ra một nhu cầu (mong muốn) đơn giản như thế cho người kỹ sư phần mềm và hy vọng có được một phần mềm chấp nhận được và dùng tốt là điều không thể. Thông tin của vấn đề cần giải quyết phải được thu thập, phân tích, phải xác định nội dung vấn đề một cách rõ ràng, đầy đủ mới có được *những yêu cầu* xác đáng cho hệ thống. Khi đó thì giải pháp phần mềm cho *những yêu cầu* này mới có thể được thiết kế và thực thi.

Quá trình thiết lập các dịch vụ mà hệ thống phải cung cấp, và các ràng buộc hệ thống phải tuân thủ khi hoạt động hay phát triển gọi là nắm bắt và phân tích yêu cầu. Kết quả của công việc này là một bản đặc tả yêu cầu, nó thường là tư liệu chính thức đầu tiên được tạo ra trong tiến trình phần mềm.

Yêu cầu cho một hệ thống phần mềm mô tả những việc mà hệ thống sẽ làm và những ràng buộc mà nó phải tuân thủ khi hoạt động. Yêu cầu có thể là yêu cầu chức năng (các chức năng, dịch vụ) hay yêu cầu phi chức năng (các ràng buộc).

Một vấn đề thường gặp khi xác định yêu cầu là sự mô tả không tách biệt rõ ràng giữa các mức của yêu cầu. Ta dùng thuật ngữ "*yêu cầu người dùng"* để mô tả yêu cầu tóm tắt ở mức cao, và thuật ngữ "*yêu cầu hệ thống"* để mô tả cụ thể công việc hệ thống sẽ làm. Ngoài hai mức trên, một mức mô tả chi tiết hơn - đặc tả thiết kế phần mềm là cầu nối giữa tiến trình xác định yêu cầu, và các hoạt động thiết kế sau này. Các yêu cầu phân theo mức trừu tượng có thể định nghĩa sau.

- Yêu cầu người sử dụng (user requirements) là những phát biểu bằng ngôn ngữ tự nhiên cùng biểu đồ để mô tả các dịch vụ mà hệ thống cung cấp, và các ràng buộc khi nó hoạt động. Đó là những phát biểu hướng người dùng.
- Yêu cầu hệ thống (system requirements) nêu ra các dịch vụ của hệ thống và chi tiết các ràng buộc của nó. Tài liệu này - đôi khi còn gọi là đặc tả chức năng và cần phải làm rõ, chính xác. Nó được sử dụng làm cơ sở cho hợp đồng giữa người mua và người phát triển.
- Đặc tả phần mềm (software specification) là sự mô tả khái quát các chức năng phần mềm trợ giúp hoạt động nghiệp vụ làm cơ sở để thiết kế, và triển khai phần mềm sau này. Tài liệu đặc tả phần mềm được bổ sung thêm các chi tiết để trở thành tài liệu đặc tả yêu cầu hệ thống.

Chẳng hạn, một ví dụ về yêu cầu người dùng và sự mở rộng, chi tiết nó để trở thành một vài yêu cầu hệ thống. Cụ thể, phần mềm phải cung cấp phương tiện biểu diễn và truy cập đến những file ngoài được tạo ra bởi một công cụ khác.

Dựa trên cơ sở yêu cầu người dùng như trên, để xác định tài liệu yêu cầu người dùng và yêu cầu hệ thống gồm các bước như sau.

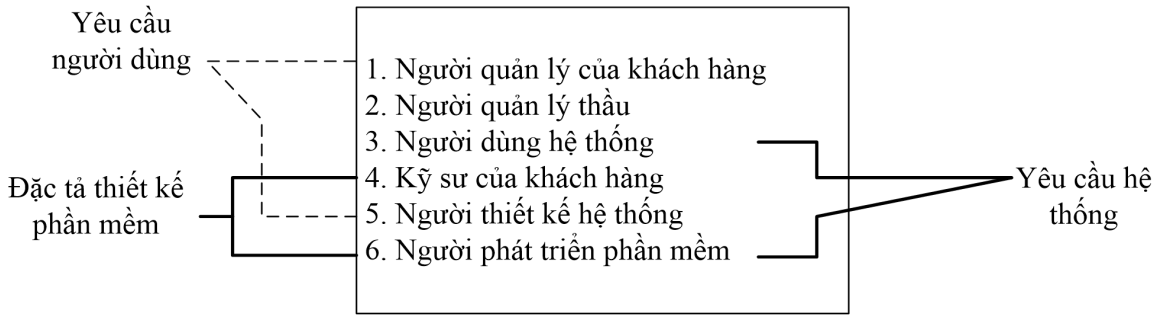
- Đầu tiên, người dùng phải được cung cấp phương tiện để dễ dàng xác định được kiểu file ngoài.
- Tiếp theo, với mỗi kiểu file ngoài cần có một công cụ thích hợp cho nó.

o Tiếp theo, mỗi kiểu file ngoài có thể được thể hiện bằng một biểu tượng (icon) trên màn hình của người dùng.

o Tiếp theo, phương tiện phải được cung cấp cho mỗi biểu tượng đại diện của một kiểu file ngoài khi người dùng xác định.

o Cuối cùng, khi người dùng chọn một biểu tượng cho kiểu file ngoài thì công cụ thích hợp được áp dụng để truy nhập và trình diễn kiểu file đó.

Mô tả yêu cầu hệ thống bằng các mức khác nhau là rất có ích vì chúng giúp truyền đạt thông tin đến người sử dụng khác nhau. Hình dưới sẽ minh họa các lớp người đọc tương ứng với các mức trừu tượng của tài liệu yêu cầu.



Hình 3. 1. Người đọc tài liệu đặc tả

Yêu cầu người dùng phải định hướng tới mức quản lý, tức là cả khách hàng và ban quản lý thầu đều hiểu được. Trong khi đó, đặc tả yêu cầu hệ thống lại hướng đến nhóm kỹ thuật và những người quản lý dự án. Người sử dụng hệ thống có thể đọc cả hai loại tài liệu nói trên. Cuối cùng, đặc tả yêu cầu phần mềm là tài liệu hướng đến việc triển khai. Nó được viết cho các kỹ sư phần mềm, những người tham gia vào quá trình phát triển hệ thống.

Yêu cầu người dùng chính là tất cả các yêu cầu về phần mềm do khách hàng, người sử dụng phần mềm đặt ra, bao gồm các chức năng của phần mềm, hiệu năng của phần mềm, các yêu cầu về thiết kế và giao diện, các yêu cầu đặc biệt khác.

Mục đích khảo sát yêu cầu người dùng nhằm xác định phần mềm sẽ đáp ứng được các yêu cầu và mong muốn của khách hàng, người sử dụng phần mềm.

Tuy nhiên, chúng ta cần lưu ý những vấn đề khi xác định yêu cầu người dùng.

o Khách hàng chỉ có những ý tưởng mơ hồ về phần mềm cần phải xây dựng phục vụ công việc của họ, chúng ta phải sẵn sàng, kiên trì theo đuổi để đi từ ý tưởng mơ hồ đó đến phần mềm có đầy đủ các tính năng cần thiết.

o Khách hàng rất hay thay đổi các yêu cầu của họ, chúng ta cần nắm bắt được các thay đổi đó và sửa đổi các mô tả một cách hợp lý.

3.1.2.Phân loại các yêu cầu phần mềm

Các yêu cầu của một hệ thống phần mềm có thể phân thành hai loại cơ bản:

o Các yêu cầu chức năng (*functional requirements*) là những phát biểu về các chức năng hay dịch vụ mà hệ thống cung cấp. Những chức năng này phụ thuộc vào loại phần mềm được xây dựng, vào sự mong đợi của người dùng, vào quy mô hệ thống được phát triển và môi trường mà ở đó nó được cài đặt. Yêu cầu chức năng cũng có thể là những chức năng mà hệ thống không cung cấp.

o Các yêu cầu phi chức năng (*non - functional requirements*) là những ràng buộc lên các dịch vụ hoặc chức năng mà hệ thống cung cấp. Nó bao gồm các ràng

buộc như về thời gian, ngân sách, các ràng buộc trong quá trình phát triển, hay các ràng buộc về chuẩn sử dụng,…

Rất nhiều yêu cầu phi chức năng liên quan đến tổng thể hệ thống hơn là đến từng bộ phận riêng rẽ. Điều đó có nghĩa, chúng thường quan trọng hơn nhiều những chức năng bộ phận, trong khi lỗi của một chức năng bộ phận có thể sửa đổi và nâng cấp, thì lỗi của một yêu cầu phi chức năng có thể làm cho hệ thống ngừng hoạt động, không sử dụng được.

Về nguyên tắc, đặc tả yêu cầu chức năng cần phải *đầy đủ* và *nhất quán*. Đầy đủ, có nghĩa là tất cả các chức năng mà người dùng cần ở hệ thống phải được xác định. Nhất quán tức là các yêu cầu nêu ra không được mâu thuẫn với nhau. Tuy nhiên, đối với một hệ thống lớn và phức tạp, điều này rất khó có thể đạt được ngay.

Đặc tả ở nhiều mức độ khác nhau rất thuận lợi vì nó phục vụ nhiều đối tượng khác nhau.

Có một vài lý do không thể hoàn chỉnh các đặc tả đối với các hệ thống.

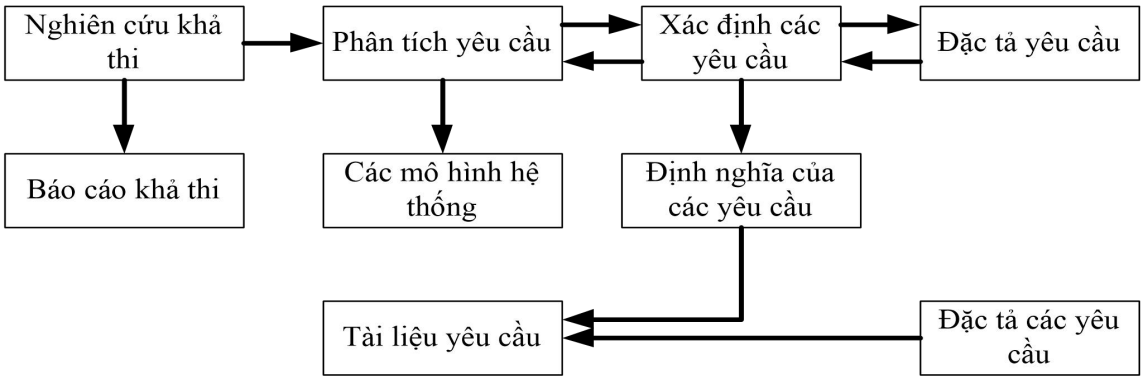
- Các hệ thống phần mềm lớn luôn đòi hỏi cải tiến từ hiện trạng. Mặc dù các khó khăn của hệ thống hiện tại có thể xác định được, nhưng các ảnh hưởng và hiệu ứng của hệ mới khó có thể thể dự đoán trước.
- Hệ thống lớn thường có nhiều cộng đồng sử dụng khác nhau. Họ có các yêu cầu và ưu tiên khác nhau. Các yêu cầu hệ thống cuối cùng không tránh khỏi các thỏa hiệp.
- Người trả tiền cho hệ thống và người sử dụng thường khác nhau. Các yêu cầu đưa ra do ràng buộc của tổ chức và tài chính có thể tranh chấp với yêu cầu của người sử dụng.

3.1.1. Quy trình khảo sát yêu cầu

Có bốn bước cơ bản trong quá trình này.

Bước 1: Nghiên cứu khả thi (*Feasibility Study*): Đánh giá xem các yêu cầu của người dùng có được thỏa mãn bởi công nghệ về phần cứng cũng như phần mềm. Nghiên cứu sẽ ra quyết định xem hệ thống có đảm bảo lợi ích so với chi phí từ quan điểm kinh tế, và tài chính cho phép thực thi hay không. Nghiên cứu khả thi nên rẻ và nhanh.

Bước 2: Phân tích yêu cầu (*Requirements Analysis*): Quá trình này suy luận các yêu cầu hệ thống qua quan sát hệ thống hiện tại, thảo luận với người sử dụng, phân tích công việc. Việc này có thể liên quan đến việc tạo một hay nhiều mô hình khác nhau. Nó giúp các phân tích viên hiểu hệ thống. Các mẫu hệ thống cũng có thể được phát triển để mô tả các yêu cầu.



Hình 3. 2. Quy trình thu thập yêu cầu

Bước 3: Xác định yêu cầu (*Requirements Definication*): Xác định yêu cầu là hoạt động biên dịch các thông tin được thu thập trong hoạt động phân tích vào tài liệu để định ra tập hợp các yêu cầu. Nó cần phản ánh chính xác các nhu cầu của người dùng. Tài liệu này cần được viết dễ hiểu cho cả người dùng và khách hàng hệ thống.

Bước 4: Đặc tả yêu cầu (*Requirements Specification*): Mô tả chi tiết và chính xác yêu cầu hệ thống được lập, tạo cơ sở cho hợp đồng giữa khách hàng và bên phát triển phần mềm. Công việc này thường được tiến hành song song với các thiết kế mức cao. Thiết kế và phân tích yêu cầu thường ảnh hưởng lẫn nhau khi được phát triển. Trong quá trình tạo lập các tài liệu, các lỗi về xác định yêu cầu sẽ được phát hiện, chúng phải được sửa để làm chính xác vấn đề.

Phân tích yêu cầu được tiếp tục xác định và đặc tả khi các yêu cầu mới nảy sinh. Do vậy, tài liệu này thường xuyên là đối tượng thay đổi và nên được kiểm soát cấu hình.

3.1.2. Tư liệu hóa phần mềm

Các yêu cầu hệ thống được trình bày trong tài liệu yêu cầu phần mềm. Nó cho biết những điều người phát triển hệ thống cần biết. Tài liệu này bao gồm các định nghĩa về yêu cầu và đặc tả các yêu cầu. Trong một số trường hợp, chúng không được trình bày riêng biệt mà được tích hợp là một. Đôi khi, định nghĩa yêu cầu được trình bày như là một giới thiệu đến đặc tả yêu cầu. Cách tiếp cận hiệu quả nhất là trình bày các đặc tả chi tiết như là phụ lục của định nghĩa yêu cầu.

Tài liệu yêu cầu phần mềm không phải là tài liệu đặc tả. Nó cần phải mô tả hệ thống cần phải làm, chứ không phải làm thế nào. Tài liệu này cần dễ dàng được đặc tả và ánh xạ sang các phần tương ứng của thiết kế hệ thống. Nếu cách dịch vụ, ràng buộc và các đặc tả thuộc tính trong tài liệu yêu cầu phần mềm được thõa mãn bởi thiết kế phần mềm, thì thiết kế này được coi như là giải pháp thích hợp với vấn đề.

Về nguyên tắc, các yêu cầu cần được hoàn chỉnh và chắc chắn. Sáu yêu cầu cần được thõa mãn.

- Nó cần mô tả các hành vi hệ thống bên ngoài.
- Nó cần mô tả các ràng buộc về thực hiện.
- Nó cần phải dễ thay đổi.
- Nó phải là công cụ tham chiếu cho người bảo trì hệ thống.
- Nó cần ghi được vòng đời của hệ thống.
- Nó cần biểu thị được các đáp ứng chấp nhận được với các sự kiện không dự kiến.

CHƯƠNG	MÔ TẢ
Giới thiệu	Mô tả sự cần thiết của hệ thống. Nó cần sự mô tả sơ lược các chức năng của mình, và giải thích cách làm việc với các hệ thống khác. Nó cũng cần mô tả làm thế nào hệ thống đáp ứng được toàn bộ các mục tiêu chiến lược và nghiệp vụ.
Thuật ngữ	Nó cần định nghĩa các khái niệm kỹ thuật được sử dụng trong tài liệu này. Không được giả định người đọc đã có kinh nghiệm.
Mô hình hệ thống	Phần này, lập một hoặc nhiều mô hình hệ thống cho biết các quan hệ giữa các cấu thành hệ thống với hệ thống và môi trường của nó. Nó cần bao gồm các mô hình đối tượng, mô hình luồng dữ liệu và ngữ

	nghĩa dữ liệu.
Định nghĩa yêu cầu chức năng	Các dịch vụ cung cấp cho người dùng cần được mô tả trong mục này. Mô tả có thể dùng ngôn ngữ tự nhiên, sơ đồ hoặc các dạng ghi chép khác cho phép khách hàng có thể hiểu được.
Định nghĩa yêu cầu phi chức năng	Các ràng buộc về phần mềm và hạn chế đối với thiết kế cần phải được mô tả trong phần này. Nó có thể bao gồm các chi tiết biểu diễn dữ liệu, thời gian đáp ứng và yêu cầu bộ nhớ,…Các tiêu chuẩn về sản phẩm và quy trình cần tuân thủ cũng được mô tả.
Tiến trình hệ thống	Phần này mô tả các giả thiết căn bản làm cơ sở cho hệ thống, và dự đoán các thay đổi về phát triển phần cứng, yêu cầu người dùng,…
Đặc tả yêu cầu	Mô tả các yêu cầu cơ bản chi tiết hơn. Nếu cần, các chi tiết hơn có thể được thêm vào các yêu cầu phi chức năng.

Bảng 3. 1. Cấu trúc chung của tài liệu yêu cầu phần mềm

3.2. Đặc tả yêu cầu

3.2.1. Khái niệm về đặc tả yêu cầu

Đặc tả yêu cầu là quá trình bổ sung các thông tin vào tài liệu xác định yêu cầu, và là sự phát triển tiếp tục của các mô hình phân tích. Đặc tả yêu cầu thường được trình bày bằng các mô hình hệ thống. Nó rất quan trọng, vì chi phí cho việc sửa đổi các yêu cầu sẽ rất cao, đặc biệt nếu các sai sót còn chưa được phát hiện trước khi hệ thống được bàn giao. Vì vậy, việc tìm ra các phương pháp và công cụ đặc tả luôn được đặt ra và không ngừng hoàn thiện.

Ngôn ngữ tự nhiên thường được sử dụng để viết đặc tả yêu cầu. Tuy nhiên, nó không phải là một ngôn ngữ đặc tả tốt do có các nguyên nhân.

- Ngôn ngữ tự nhiên không hoàn toàn chính xác.
- Ngôn ngữ tự nhiên vốn quá mềm dẻo.
- Ngôn ngữ tự nhiên không thể phân chia yêu cầu một cách hiệu quả.

Do những hạn chế của nó, nhiều lựa chọn khác đã được nghiên cứu thay cho việc đặc tả bằng ngôn ngữ tự nhiên.

- Ngôn ngữ tự nhiên có cấu trúc.
- Ngôn ngữ mô tả chương trình.
- Ngôn ngữ đặc tả yêu cầu.
- Đặc tả dựa trên mô hình với cú pháp đồ họa.
- Đặc tả bằng toán học - hình thức hóa.

3.2.2. Các ngôn ngữ đặc tả yêu cầu

3.2.2.1. Ngôn ngữ tự nhiên có cấu trúc

Ngôn ngữ tự nhiên có cấu trúc là một dạng mở rộng của ngôn ngữ tự nhiên. Nó phụ thuộc vào các định dạng, hoặc mẫu chuẩn được sử dụng để biểu diễn yêu cầu. Bảng quyết định của Moret (1992) là một dạng của ngôn ngữ có cấu trúc. Chúng là sự kết hợp các cấu trúc điều khiển thu được từ ngôn ngữ lập trình, và các hình vẽ nổi bật để phân chia đặc tả. Lợi thế của cách tiếp cận này là nó giữ được phần lớn những đặc điểm dễ hiểu của ngôn ngữ tự nhiên, lại có cấu trúc rõ ràng, chính xác của ngôn ngữ lập trình.

3.2.2.2. Ngôn ngữ mô tả chương trình - PDL

Ngôn ngữ mô tả chương trình - PDL (*Program Decription Language*) là tương tự với một ngôn ngữ lập trình nào đó nhưng trừu tượng hơn, có nhiều tính năng đặc tả hơn ngôn ngữ tự nhiên có cấu trúc. Ưu điểm của nó là có thể kiểm tra lỗi cú pháp và ngữ nghĩa bằng các công cụ phần mềm. Những mâu thuẫn và sai sót có thể được phát hiện nhờ việc kiểm tra này.

Người đọc có hiểu biết về ngôn ngữ lập trình có thể đọc hiểu các đặc tả yêu cầu bằng PDL một cách dễ dàng. Nếu PDL dựa trên một ngôn ngữ thực thi cụ thể thì quá trình chuyển đổi từ đặc tả sang thiết kế là rất tự nhiên và đơn giản.

Chẳng hạn, chương trình tính tổng n số tự nhiên đầu tiên nhập từ bàn phím, cụ thể.

```
public class Sumn {
        public static void main(String[] args) {
            Scanner input = new Scanner(System.in);
            System.out.println("Nhập n: ");
            int n = input.nextInt();
            int s = 0;
            int i = 0;
            do{
                s = s+i; i++;
            }while(i<=n);
            System.out.println("Tổng n số tự nhiên là S = "+s);
        }}
```

Hình 3. 3. Chương trình tính tổng n đầu tiên nhập từ bàn phím

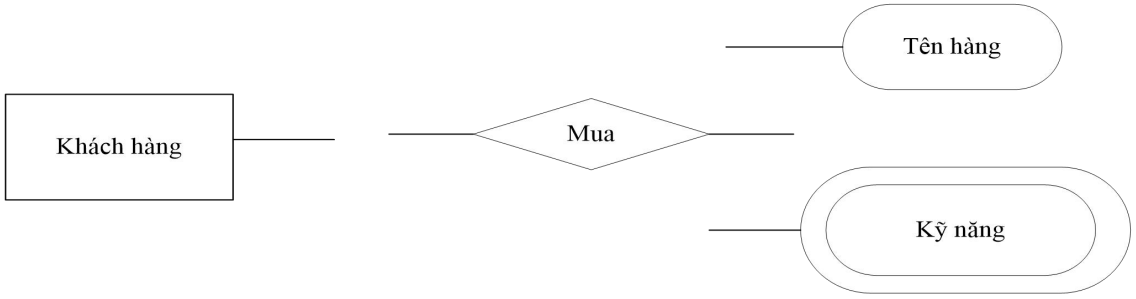
3.2.3. Đặc tả yêu cầu dựa trên mô hình

3.2.3.1. Mô hình thực thể mối quan hệ

Mô hình thực thể - mối quan hệ ERM (*Entity - Relationships Model*) còn có thể được xem như đặc tả dữ liệu hướng cấu trúc, được Chen đề xuất năm 1976, sau đó được Chouinard (1989) bổ sung và mở rộng. Nó mô tả dữ liệu của thế giới thực đúng như chúng đang tồn tại. Nó cũng là phương tiện để giao tiếp với người sử dụng giúp xác định và mô tả đúng đắn, đầy đủ yêu cầu thông tin của hệ thống. Mô hình được viết ngắn gọn là ERM.

Có nhiều mô hình cho phép biểu diễn các đối tượng hệ thống. Trong đó, phải kể đến mô hình thông dụng như mô hình thực thể liên kết, mô hình hướng đối tượng. Chúng ta chỉ xét mô hình thực thể mối quan hệ. Mô hình sử dụng một số khái niệm chính như.

- Thực thể (*Entity*): Là khái niệm chỉ một lớp các đối tượng hay khái niệm (độc lập) của thế giới thực có những đặc trưng chung.
- Quan hệ (*Relationship*): Là khái niệm chỉ mối quan hệ vốn có giữa các bản thể của các thực thể. Tên của nó là động từ chỉ sự tương tác hay mối quan hệ phụ thuộc, sở hữu giữa các bản thể của các thực thể.
- Thuộc tính (*Attribute*): Là đặc trưng của thực thể hay mối quan hệ của thực thể.

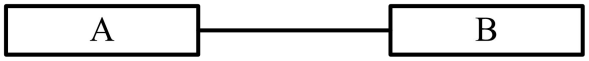


a. Thực thể b. Mối quan hệ c. Thuộc tính

Hình 3. 4. Các ký pháp sử dụng trong ERM

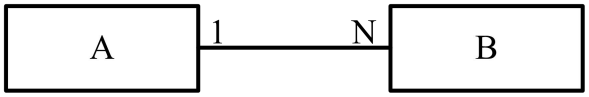
Người ta thường sử dụng 3 loại quan hệ để biểu diễn mối quan hệ giữa các đối tượng dữ liệu.

o Quan hệ 1 - 1: Một thể hiện của kiểu thực thể A ứng với một thể hiện của kiểu thực thể B. Quan hệ này có thể biểu diễn bởi 1 đoạn thẳng nối 2 kiểu thực thể.



Hình 3. 5. Biểu diễn quan hệ 1-1

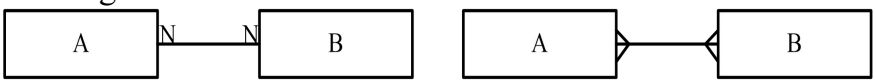
o Quan hệ một nhiều 1 - N: Một thể hiện của kiểu thực thể A ứng với 0, 1, 2,… nhiều thể hiện của kiểu thực thể B. Cách biểu diễn như sau:



Hình 3. 6. Biểu diễn quan hệ 1 - N

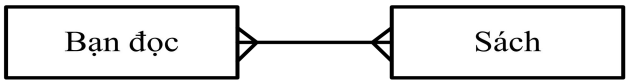
Ví dụ, một phòng có nhiều nhân viên.

o Quan hệ nhiều nhiều N - N: Một thể hiện của A ứng với nhiều thể hiện của kiểu thực thể B và ngược lại, một thể hiện của kiểu thực thể B ứng với nhiều thể hiện của kiểu thực thể A. Người ta biểu diễn nó như sau:



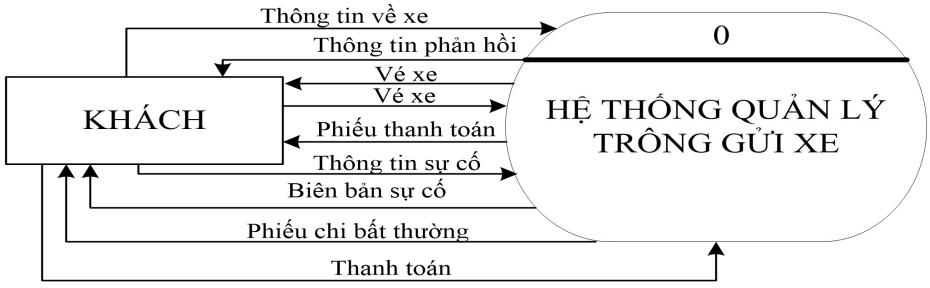
Hình 3. 7. Biểu diễn quan hệ nhiều - nhiều

Chẳng hạn như hình minh họa sau:



Hình 3. 8. Biểu diễn quan hệ bạn đọc - sách

Ví dụ, cho mô hình hình nghiệp vụ minh họa mô tả của hoạt động trông gửi xe ở một bãi được thể hiện hình bên dưới. Hãy xác định mô hình ERM?



Hình 3. 9. Mô hình hệ thống trông gửi xe

STT	Các mục tin	STT	Các mục tin	STT	Các mục tin
1.	Số vé	5.	Số phiếu thông tin	9.	Tên loại xe
2.	Số xe	6.	Ngày ra	10.	Số chỗ
3.	Ngày vào	7.	Giờ ra	11.	Giá gửi loại xe
4.	Giờ vào	8.	Thành tiền		

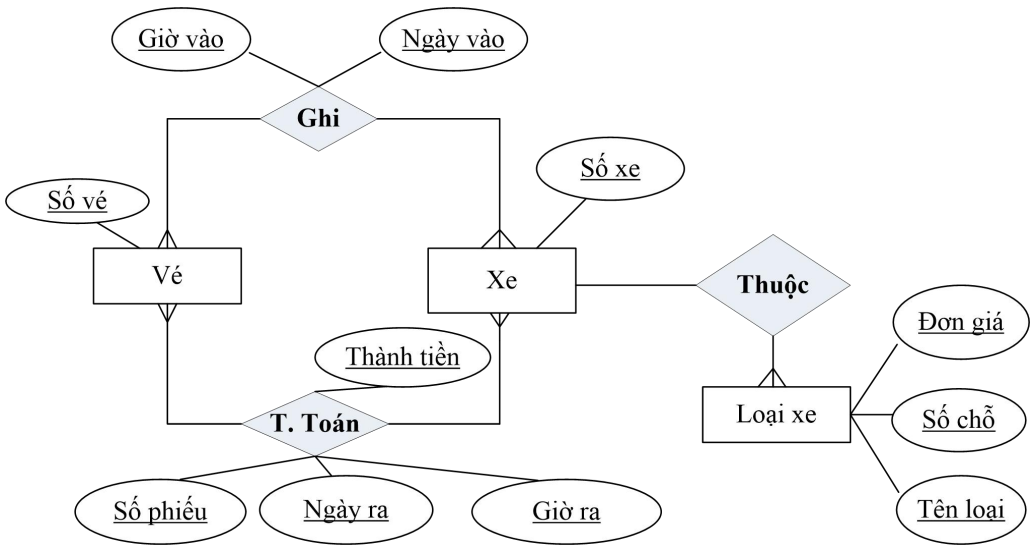
Bảng 3. 2. Các mục tin cơ sở

Thực thể	Thuộc tính	Thực thể	Thuộc tính
Vé	Số vé	Loại xe	Tên loại xe
Xe	Số xe		Số chỗ (trong bãi)
			Đơn giá gửi

Bảng 3. 3. Các thực thể và thuộc tính

Quan hệ	Thực thể tham gia và thuộc tính	Quan hệ	Thực thể tham gia và thuộc tính
1. Ghi	Vé, Xe	2. Thanh toán	Vé, Xe
	Ngày vào		Số phiếu
	Giờ vào		Ngày ra
			Giờ ra
			Thành tiền
3. Thuộc	Xe, Loại xe		

Bảng 3. 4. Các quan hệ và thuộc tính của nó

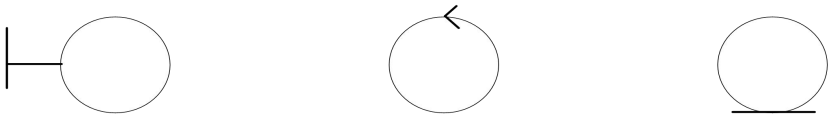


Hình 3. 10. Mô hình ERM của hệ thống trông gửi xe

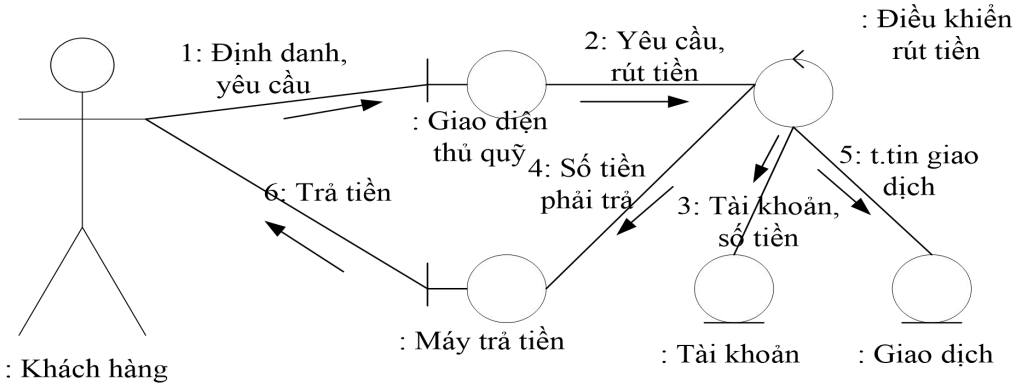
3.2.3.2. Biểu đồ phân tích tương tác

Thực thi ca sử dụng phân tích là một mô tả về sự cộng tác giữa các phần tử trong mô hình để thực hiện chức năng của một ca sử dụng cụ thể, và thể hiện ra dưới dạng

các lớp phân tích, và các đối tượng của nó tương tác với nhau. Các lớp phân tích có ba dạng cơ bản: *lớp biên, lớp điều khiển* và *lớp thực thể.*



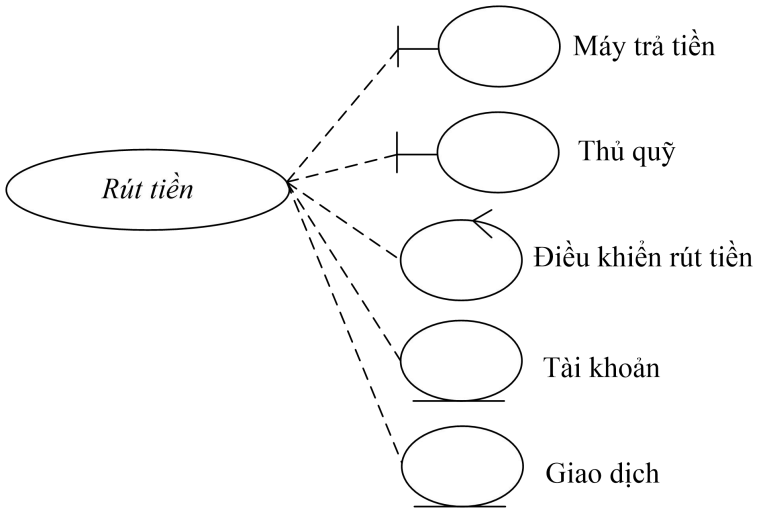
Hình 3. 11. Các lớp phân tích



Hình 3. 12. Biểu đồ cộng tác giữa các lớp thực thi ca sử dụng rút tiền

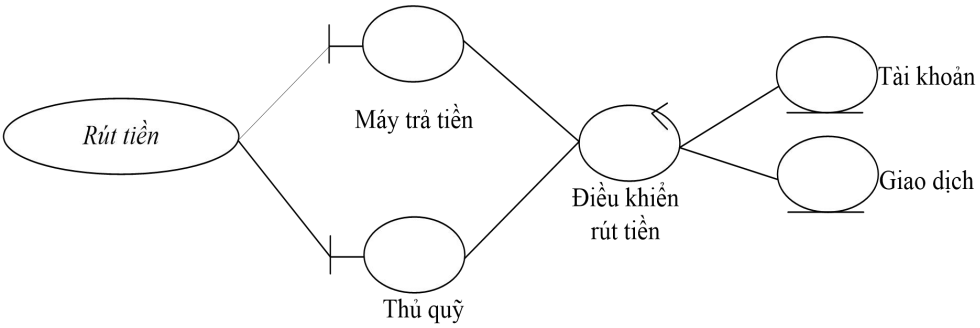
Trước hết cần xác định các lớp phân tích bằng cách làm mịn ca sử dụng trên luồng các sự kiện mô tả ca sử dụng bằng văn bản, mô hình miền sử dụng.

Dựa trên hình 3.12, chúng ta xác định các lớp phân tích đối với ca sử dụng rút tiền được thể hiện bởi hình sau.



Hình 3. 13. Các lớp phân tích tham gia vào thực thi cả sử dụng rút tiền

Xác định các *mối liên kết giữa các lớp phân tích và tác nhân*, cụ thể dựa vào hình 3.12, ta thể hiện mối liên kết như hình sau.



Hình 3. 14. Quan hệ giữa các lớp phân tích và tác nhân của ca sử dụng *rút tiền*

Phân tích để xác định các trách nhiệm của các lớp trong sự cộng tác thực thi ca sử dụng. Ví dụ, lớp *giao diện thủ quỹ* có các trách nhiệm.

- Hiển thị bảng tiếp nhận yêu cầu.
- Tiếp nhận các yêu cầu của khách (hay từ lớp khác gửi đến).
- Chuyển yêu cầu đến lớp điều khiển (hay đến một lớp khác).

Đây là các trách nhiệm điển hình của một giao diện người dùng dễ nhận ra.

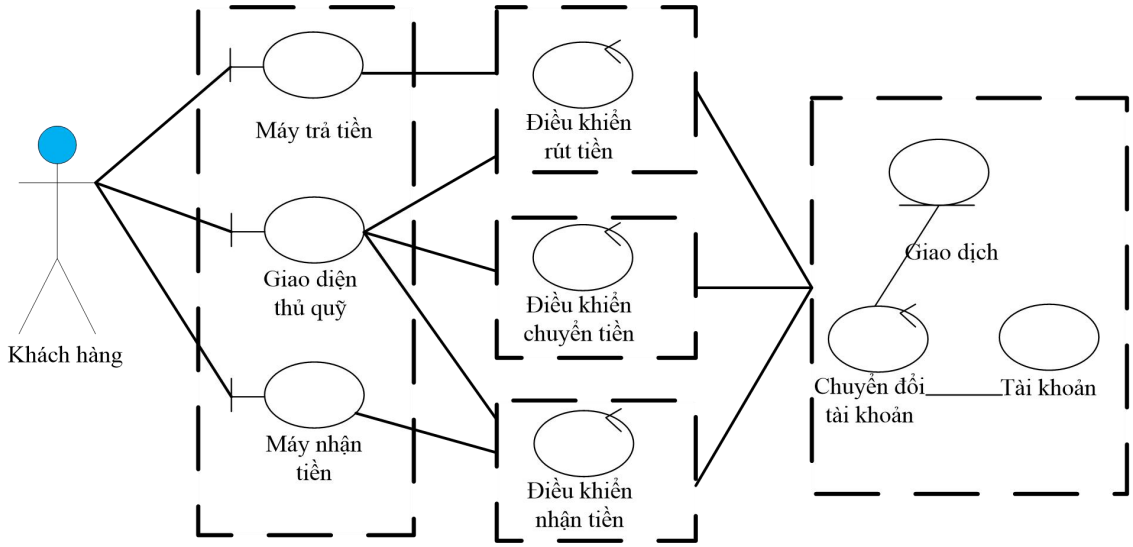
Xác định các *thuộc tính của lớp*. Một thuộc tính đặc tả một tính chất của một lớp phân tích, và nó thường được gợi ý và yêu cầu trách nhiệm của mỗi lớp.

Xác định các *liên kết và các kết hợp sao cho hợp lý*. Số lượng các mối quan hệ giữa các lớp phải được tối thiểu.

Nắm bắt các *yêu cầu đặc biệt của lớp phân tích*. Ví dụ, các đặc trưng yêu cầu về tính lâu bền của lớp tài khoản có thể định chuẩn.

- Quy mô: 1 đến 4KB cho mỗi đối tượng.
- Số lượng: Tối đa 100.000 tài khoản.

Phân tích để *nhóm các lớp thành gói*. Mục đích của việc phân tích càng độc lập đối với các gói khác càng tối ưu.



Hình 3. 15. Mô hình lớp phân tích đặc tả hệ thống với các gói

3.3. Định dạng tài liệu yêu cầu

3.3.1. Nội dung và yêu cầu đối với tài liệu yêu cầu phần mềm

Yêu cầu hệ thống được biểu diễn bằng *tài liệu yêu cầu phần mềm* (hay đặc tả yêu cầu phần mềm - *Software Requirements Specification* - *SRS*). Nó là tài liệu chính thức

cho người phát triển phần mềm. Tài liệu này bao gồm *tài liệu xác định yêu cầu* và *tài liệu đặc tả yêu cầu*. Có thể được trình bày hai tài liệu đó trong một bản mô tả chung, trong đó phần đặc tả như là phụ lục của tài liệu xác định yêu cầu. Nó được dùng để viết bản thiết kế cuối cùng. Vì thế, từ một yêu cầu phải tham chiếu được tới các phần thiết kế tương ứng để thực thi nó. Nếu các dịch vụ, ràng buộc và thuộc tính trong tài liệu yêu cầu phù hợp với bản thiết kế phần mềm, thì bản thiết kế đó là lời giải đúng cho bài toán đặt ra.

Heniger (1980) đưa ra sáu yêu cầu cho một tài liệu yêu cầu phần mềm.

- Chỉ mô tả các hoạt động của hệ thống từ bên ngoài.
- Chỉ ra được các ràng buộc của hệ thống trong quá trình vận hành.
- Dễ thay đổi.
- Phục vụ như tài liệu tham khảo cho người bảo trì hệ thống.
- Dự toán trước được vòng đời của hệ thống.
- Mô tả được các đáp ứng đối với những sự cố, thay đổi ngoài dự tính.

Về nguyên tắc, yêu cầu được xác định phải là *đầy đủ và nhất quán*. Nhưng việc tồn tại lỗi và thiếu sót trong xác định yêu cầu là điều không thể tránh khỏi. Vì thế, tài liệu cần được cấu trúc sao cho sau này dễ sửa đổi mà không cần viết lại. Quá trình quản lý thay đổi yêu cầu được trợ giúp bởi các hệ thống phần mềm quản lý thay đổi cấu hình phần mềm, trong đó có sự thay đổi về yêu cầu phần mềm.

3.3.2. Cấu trúc tài liệu yêu cầu phần mềm

Các tổ chức lớn như Bộ quốc phòng Mỹ hay IEEE đã xác định các chuẩn cho tài liệu yêu cầu. Chuẩn IEEE/ANSI 830 - 1993 đã gợi ý một cấu trúc cho tài liệu yêu cầu như sau.

1. Giới thiệu
   1.1. Mục tiêu của tài liệu yêu cầu
   1.2. Phạm vi của sản phẩm
   1.3. Các định nghĩa, chữ viết tắt và từ viết rút gọn
   1.4. Tài liệu tham khảo
   1.5. Tổng quan về tài liệu
2. Mô tả chung
   2.1. Các đặc điểm của sản phẩm
   2.2. Các chắc năng của sản phẩm
   2.3. Các đặc trưng người dùng
   2.4. Các ràng buộc chung
   2.5. Các giả định và các sự phụ thuộc
3. Các yêu cầu cụ thể
4. Phụ lục
5. Các chỉ số chỉ dẫn

Tài liệu yêu cầu cũng là một công cụ tham khảo. Nó phải bao gồm cả dự đoán vòng đời của hệ thống. Các kỹ sư bảo trì dùng tài liệu này để xác định hệ thống cần làm gì trong vòng đời của nó. Khả năng lần vết là một ưu điểm của đặc tả yêu cầu vì nó cho phép dễ dàng tìm ra các yêu cầu liên quan. Các kỹ thuật từ đơn giản đến các công cụ CASE được dùng để hỗ trợ chức năng này.

3.4. Câu hỏi và bài tập

Câu 1. Yêu cầu phần mềm là gì? Phân biệt giữa yêu cầu và nhu cầu? Có thể phân yêu cầu theo những đối tượng đọc nào và đặc trưng của mỗi yêu cầu?

Câu 2. Đối tượng chính của tài liệu yêu cầu là ai? Những yêu cầu gì đặt ra để viết bản tài liệu yêu cầu? Vì sao?

Câu 3. Trình bày các bước của tiến trình phát triển và phân tích yêu cầu?

Câu 4. Cấu trúc tài liệu gồm những mục gì?

Câu 5. Đặc tả yêu cầu là gì? Cần phải đặc tả những nội dung gì phần mềm?Những kỹ thuật gì được sử dụng để đặc tả yêu cầu?

Câu 6. Hạn chế của đặc tả ngôn ngữ tự nhiên? Ngôn ngữ tự nhiên có cấu trúc là gì? Ưu và nhược điểm của nó?

Câu 7. Đặc tả bằng ngôn ngữ mô tả chương trình là gì? Ưu điểm của nó?

Câu 8. Sự khác nhau giữa phần mềm dùng chung(generic software) và phần mềm đặt hàng(custom software) là gì?

Câu 9. Cấu trúc cơ bản của tài liệu yêu cầu gồm những mục gì?

Câu 10. Case là gì? Nêu tầm quan trọng của CASE trong phát triển phần mềm?

CHƯƠNG 4. THIẾT KẾ PHẦN MỀM

4.1. Tổng quan về thiết kế phần mềm

4.1.1. Khái niệm và vai trò của thiết kế

4.1.1.1. Khái niệm về thiết kế

Quá trình xây dựng các ứng dụng là một dây chuyền các chuyển đổi. Phân tích là nghệ thuật nhằm xác định một ứng dụng sẽ thực hiện cái gì (*what*). Thiết kế lại là nghệ thuật xác định cách thức thực hiện những yêu cầu đã được đặt ra trong phần phân tích, trong một môi trường phần cứng và phần mềm cụ thể như thế nào (*how*).

Thiết kế là bước đầu tiên trong giai đoạn phát triển bất cứ một sản phẩm hay hệ thống công nghệ nào. Nó được định nghĩa là "*tiến trình áp dụng nhiều kỹ thuật, và nguyên lý với mục đích xác định một thiết bị, một tiến trình hay một hệ thống đủ chi tiết để cho phép thực hiện nó về mặt vật lý*".

Thiết kế phần mềm là quá trình chuyển hóa các đặc tả yêu cầu phần mềm thành một biểu diễn thiết kế của hệ thống phần mềm cần xây dựng, sao cho người lập trình có thể ánh xạ nó thành chương trình. Trong trường hợp chung, các yêu cầu phần mềm thường được mô tả ở *mức quan niệm*, tức là mô tả các dịch vụ mà phần mềm cung cấp cũng như các ràng buộc nó cần tuân thủ khi hoạt động. Nhiệm vụ của người kỹ sư là chuyển những yêu cầu này thành một mô tả về hệ thống phần mềm, với các giải pháp công nghệ thích hợp sẽ vận hành trên thực tế nhằm đáp ứng các yêu cầu đặt ra. Để làm được điều đó, người kỹ sư thiết kế cần đến một số hoạt động chính.

- Nghiên cứu để hiểu vấn đề: Vấn đề ở đây không chỉ liên quan đến yêu cầu đặt ra, mà còn liên quan đến môi trường đích mà ở đó hệ thống được cài đặt và vận hành.
- Chọn một số giải pháp thiết kế và xác định các đặc điểm thô: Các giải pháp được chọn cần khả thi và cho hiệu quả cao đối với hệ thống.
- Mô tả trừu tượng cho mỗi giải pháp thiết kế: Trước hết cần xây dựng một mô tả ban đầu sơ khai rồi chi tiết hóa dần. Các sai sót ở mỗi mức thiết kế trước đó được phát hiện và chỉnh sửa trước khi lập tư liệu thiết kế chính thức.

Quá trình thiết kế nhằm tạo ra một mô hình sản phẩm mới bằng việc tìm tòi và vận dụng các giải pháp công nghệ khác nhau. Hoạt động thiết kế vì thế là một hoạt động đặc biệt. Một quá trình sáng tạo, đòi hỏi kinh nghiệm và sự nhanh nhạy của người thiết kế. Kinh nghiệm cho thấy, nó phải được thực hành, được học bằng kinh nghiệm thực tiễn, bằng khảo sát các hệ đang tồn tại, không thể chỉ học qua sách vở.

4.1.1.2. Vai trò của thiết kế

Thiết kế là chuyển yêu cầu phần mềm thành các đặc tả thiết kế mô tả một hệ thống phần mềm mới sẽ xây dựng.

- Thiết kế là cách duy nhất để chuyển hóa một cách chính xác các yêu cầu của khách hàng thành mô hình thiết kế hệ thống phần mềm cuối cùng, làm *cơ sở cho việc triển khai chương trình phần mềm*.
- Tài liệu thiết kế phần mềm là *công cụ giao tiếp* giữa các nhóm cùng tham gia vào việc phát triển sản phẩm, *để quản lý các rủi ro*, *đạt đến phần mềm hiệu quả*.
- Thiết kế phần mềm là tài liệu *cung cấp đầy đủ các thông tin* cần thiết cho những người kỹ sư hệ thống *để bảo trì hệ thống* sau này.

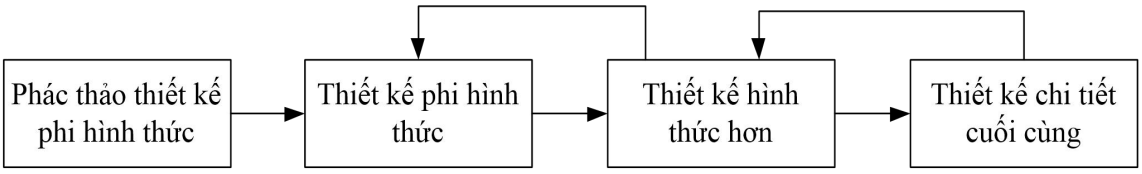
Không có thiết kế có nguy cơ sản sinh một hệ thống không tin cậy - một hệ thống dễ thất bại. Với bản chất không nhìn thấy và phức tạp, một hệ thống phần mềm rất khó xác định chất lượng chừng nào chưa được kiểm thử, chưa vận hành, nên thiết kế tốt là

chìa khóa làm cho hệ thống phần mềm hữu hiệu. Không thể hình thức hóa mọi quá trình thiết kế đối với bất kỳ một sản phẩm phần mềm nào.

4.1.2. Triển khai thiết kế

4.1.2.1. Tiến trình thiết kế

Tiến trình thiết kế (*Design Process*) là quá trình tăng cường hình thức hóa trong sự tiến triển của thiết kế, và phải luôn quay trở lại các thiết kế đúng đắn ít hình thức có trước của quá trình này. Tiến trình thiết kế có thể được mô tả như sau.



Hình 4. 1. Mô hình tiến trình thiết kế

Kết quả của mỗi hoạt động thiết kế là một đặc tả. Đặc tả này có thể là một đặc tả trừu tượng, bán hình thức, hình thức hay cũng có thể là một đặc tả về một phần nào đó của hệ thống phải được thực hiện như thế nào. Khi quá trình thiết kế tiến triển thì các chi tiết được bổ sung dần vào đặc tả thiết kế. Các kết quả cuối cùng là các đặc tả về các thuật toán, các cấu trúc dữ liệu và đặc tả giao diện được dùng làm cho cơ sở việc mã hóa.

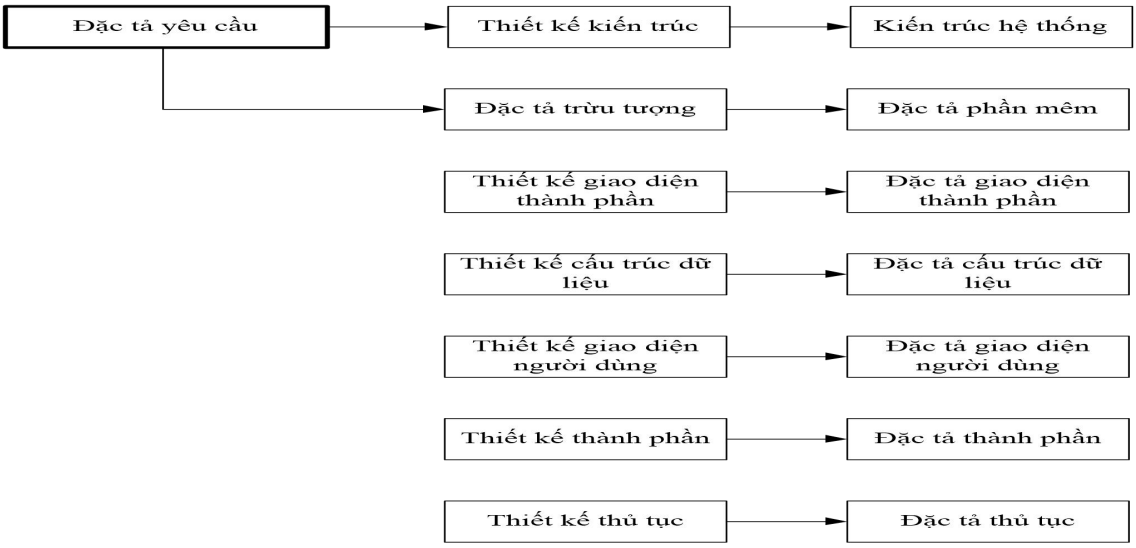
Tiến trình phát triển phần mềm là một sự hoàn thiện liên tục, nên *giữa thiết kế và đặc tả yêu cầu có mối quan hệ rất chặt chẽ với nhau*. Thực thể, người thiết kế sẽ phải lặp đi lặp lại giữa các đặc tả yêu cầu và thiết kế. Vì rằng, khi ở mức ít trừu tượng hơn, người ta dễ phát hiện được những sai sót của các giai đoạn trước, do đó phải quay lại để chỉnh sửa sản phẩm trước đó. Hình 4.2 đưa ra các hoạt động của tiến trình thiết kế và các sản phẩm của nó. Các hoạt động này có thể là tùy biến, nhưng việc xác định nó làm cho tiến trình thiết kế trở nên nhìn thấy được và dễ quản lý.

Phương pháp tiếp cận để phát triển hệ thống thường được dùng là *phương pháp tiếp cận từ trên xuống*. Tức là, vấn đề được phân chia một cách đệ quy thành các vấn đề con, cho đến khi các vấn đề nhận được có thể giải quyết một cách dễ dàng. Khi thiết kế thường được thực hiện *từ dưới lên*, vì rằng các vấn đề ở mức thấp nhất là đủ nhỏ, đủ thông tin cần cho nó, và cũng nhận ra các thành phần nào có thể kế thừa các phần khác.

4.1.2.2. Các hoạt động và sản phẩm thiết kế

Thiết kế một hệ thống phần mềm lớn thường bao gồm các hoạt động được minh họa cụ thể bởi hình 4.2.

- Thiết kế kiến trúc: Xác định các hệ con tạo nên hệ thống tổng thể và các mối quan hệ giữa chúng.



Hình 4. 2. Các hoạt động thiết kế và sản phẩm của chúng

o Đặc tả trừu tượng: Đối với mỗi hệ con, cần mô tả trừu tượng các dịch vụ mà nó cung cấp cùng các ràng buộc mà nó phải tuân thủ khi cung cấp dịch vụ.

o Thiết kế giao diện thành phần: Thiết kế giao diện của từng hệ con với các hệ con khác, các hệ thống khác sao cho có thể sử dụng các hệ con thông qua giao diện, mà không cần biết nó sẽ thực hiện như thế nào, và nhờ vậy có thể phát triển các hệ con một cách độc lập.

o Thiết kế cấu trúc dữ liệu: Thiết kế cấu trúc dữ liệu lưu trữ và đặc tả các cấu trúc dữ liệu được dùng trong việc thực hiện hệ thống.

o Thiết kế hệ thống giao diện người dùng: Thiết kế các giao diện bên ngoài mà qua đó người dùng có thể sử dụng được hệ thống.

o Thiết kế thành phần: Phân chia các dịch vụ mà một hệ thống con cung cấp vào các thành phần hợp thành của nó.

o Thiết kế thủ tục: Thiết kế đặc tả các thuật toán, quy trình dùng để thực hiện các dịch vụ của mỗi thành phần sao cho có thể ánh xạ trực tiếp nó vào một ngôn ngữ lập trình.

4.1.2.3. Biểu diễn thiết kế

Một bản thiết kế phần mềm là một mô hình mô tả một đối tượng của thế giới thực với nhiều thành phần, các mối quan hệ giữa chúng với nhau. Có ba hình thức thường được sử dụng để mô tả thiết kế.

o Biểu đồ: Có các biểu đồ được dùng để thể hiện các mối quan hệ giữa các thành phần lập nên hệ thống, đồng thời là mô hình mô tả thế giới thực. Việc mô tả đồ thị của các thiết kế là rất có lợi vì tính trực quan, và cho một bức tranh tổng thể về hệ thống.

o Ngôn ngữ mô tả chương trình: Các ngôn ngữ này dùng để kiểm tra cấu trúc các cơ cấu thiết kế dựa trên các cấu trúc của một ngôn ngữ lập trình.

o Dạng văn bản không hình thức hóa: Loại hình này mô tả các thông tin không thể hình thức hóa được như các thông tin phi chức năng bên cạnh các cách mô tả khác.

4.1.3. Các chiến lược và phương pháp thiết kế

Việc thiết kế phần mềm ở nhiều tổ chức vẫn còn là một quá trình tự học. Khi cho một tập hợp các yêu cầu (thường bằng ngôn ngữ tự nhiên) người ta dễ dàng có ngay một phác họa thiết kế không hình thức. Từ đó, bắt đầu việc mã hóa và thiết kế được

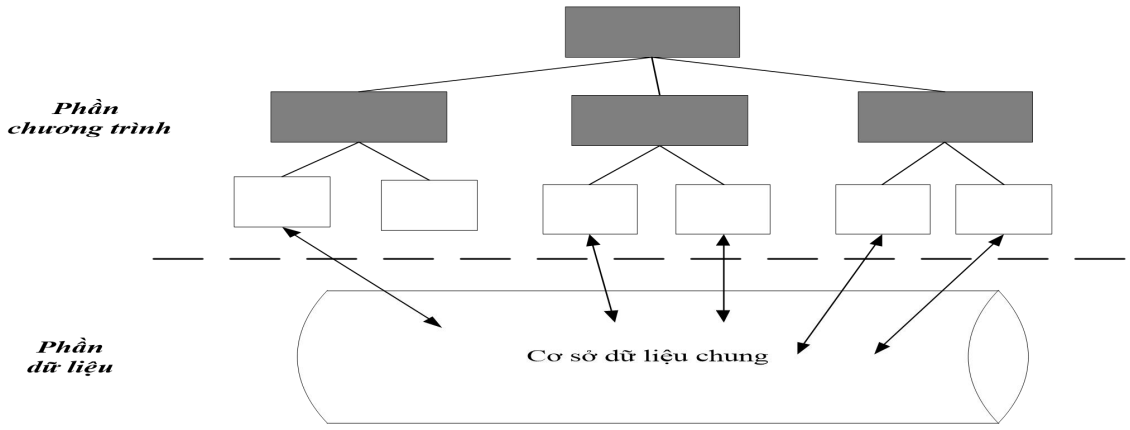
cải biên trong hệ thống khi được hình thành. Kết thúc giai đoạn triển khai, thì sản phẩm thiết kế đã biến đổi khác xa so với đặc tả ban đầu đến mức mà các tài liệu thiết kế nguyên thủy là một mô tả hoàn toàn khác với hệ thống được đưa ra.

Hiện nay có rất nhiều phương pháp tiếp cận khác nhau được áp dụng cho thiết kế. Nó làm cho quá trình thiết kế trở nên rõ ràng hơn, có thể theo dõi được và mang tính khoa học nhiều hơn. Hai cách tiếp cận phổ biến hiện nay là *tiếp cận định hướng cấu trúc (hay chức năng) và tiếp cận định hướng đối tượng*. Tương ứng, mỗi cách tiếp cận này là *một chiến lược cho việc phát triển hệ thống phần mềm*. Với mỗi chiến lược, do đặc trưng của cách tiếp cận và đặc thù của bài toán mà các phương pháp được sử dụng có thể là rất khác nhau.

4.1.3.1. Chiến lược và phương pháp hướng cấu trúc

Theo cách tiếp cận định hướng cấu trúc, hệ thống được phân chia thành các chức năng, bắt đầu ở mức cao nhất, sau đó làm mịn dần dần để thành thiết kế với các chức năng chi tiết hơn. Trạng thái của hệ thống thể hiện qua cơ sở dữ liệu chung và được chia sẻ cho các chức năng thao tác trên nó.

Với đặc trưng trên, thiết kế chức năng thường có hai hoạt động tương đối độc lập *thiết kế dữ liệu và thiết kế xử lý*. Sau giai đoạn phân tích - đặc tả yêu cầu, thiết kế dữ liệu thường đi trước một bước để có cơ sở dữ liệu dùng chung (độc lập với chương trình). Các thiết kế xử lý (các chương trình) được thực hiện trên cơ sở sử dụng các cấu trúc dữ liệu lưu trữ đã có.



Hình 4. 3. Mô hình hệ thống hướng cấu trúc

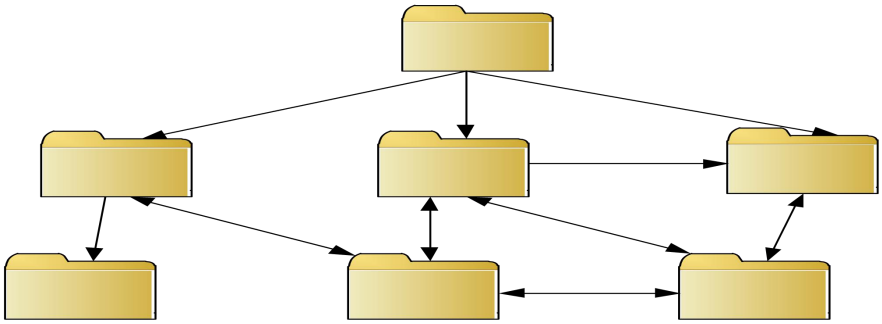
Vào cuối những năm 1960, Dijkstra và cộng sự đã đề nghị việc dùng một tập các cấu trúc logic đơn giản để từ đó tạo nên bất kỳ chương trình nào. Các cấu trúc đều nhấn mạnh vào "*duy trì miền chức năng*". Tức là mỗi cấu trúc đều có một cấu trúc logic dự đoán được, có lối vào trên đỉnh, có lối ra ở đáy, làm cho độc giả theo dõi được luồng thủ tục dễ dàng.

Ba cấu trúc tiêu biểu của lập trình cấu trúc được Bolun (1966) nêu ra là *tuần tự, tuyển chọn và lặp*. Cấu trúc *tuần tự* dành cho các bước xử lý theo trình tự lần lượt mà về bản chất có trong đặc tả của bất kỳ các thuật toán. Cấu trúc *tuyển chọn* dành cho việc xử lý có sự lựa chọn dựa trên một sự kiện logic, đối với cấu trúc *lặp* thích hợp cho mọi quá trình lặp lại.

4.1.3.2. Chiến lược và phương pháp hướng đối tượng

Theo hướng đối tượng, hệ thống được nhìn nhận như một bộ các đối tượng tương tác với nhau. Mỗi đối tượng bao hàm trong nó cả dữ liệu và các thao tác thực hiện trên chúng, vì thế nó có trạng thái riêng. Mỗi đối tượng là một thể hiện của một lớp được

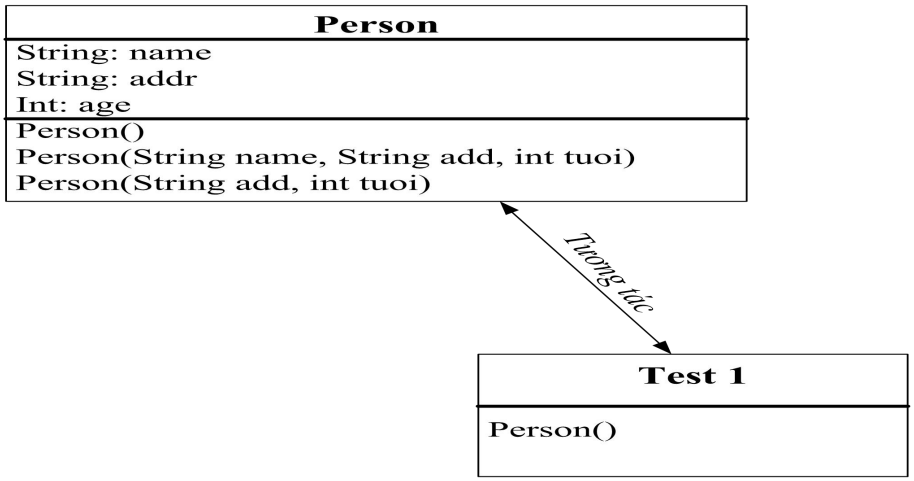
xác định bởi *các thuộc tính* và *các phương thức*. Bộ các *thuộc tính* xác định trạng thái của đối tượng và các phương thức nó thao tác với các dữ liệu của thuộc tính. Các lớp có thể được thừa kế từ một vài lớp đối tượng cao hơn, và khi định nghĩa nó chỉ cần nêu đủ những gì khác nhau giữa nó và các lớp cha. Các đối tương liên lạc với nhau chỉ bằng cách trao đổi các thông báo để cộng tác thực hiện một dịch vụ chung.



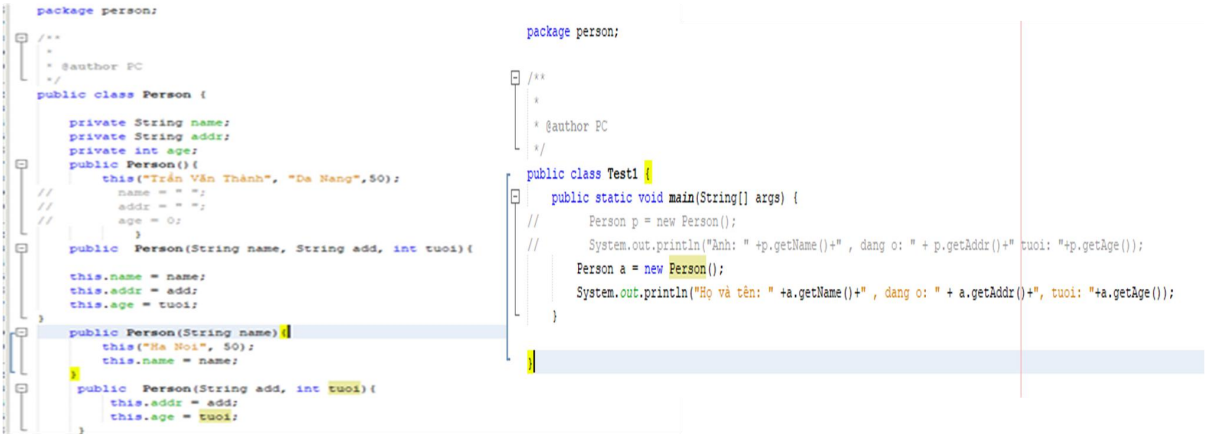
Hình 4. 4. Mô tả cấu trúc của hệ thống hướng đối tượng

Mặc dù mới phát triển từ đầu những năm 90 trở lại gần đây, có nhiều phương pháp phát triển phần mềm hướng đối tượng ra đời. Một ngôn ngữ chung có tính chuẩn cao UML cùng nhiều công cụ mạnh đã được phát triển để trợ giúp cho các phương pháp theo định hướng này.

Cụ thể minh họa bởi chương trình sau:



Hình 4. 5. Minh họa tính kế thừa từ 2 lớp



```
package person;
/**
 *
 * @author PC
 */
public class Person {
    private String name;
    private String addr;
    private int age;
    public Person(){
        this("Trần Văn Thành", "Da Nang",50);
//        name = " ";
//        addr = " ";
//        age = 0;
        }
    public  Person(String name, String add, int tuoi){
    this.name = name;
    this.addr = add;
    this.age = tuoi;
}
    public Person(String name){
        this("Ha Noi", 50);
        this.name = name;
    }
     public  Person(String add, int tuoi){
         this.addr = add;
         this.age = tuoi;
     }
```

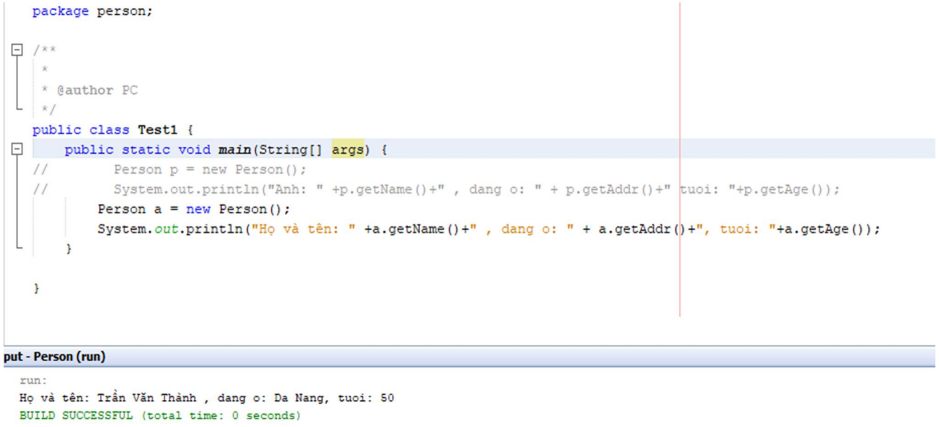
```
package person;

/**
 *
 * @author PC
 */
public class Test1 {
    public static void main(String[] args) {
//        Person p = new Person();
//        System.out.println("Anh: " +p.getName()+" , dang o: " + p.getAddr()+" tuoi: "+p.getAge());
        Person a = new Person();
        System.out.println("Họ và tên: " +a.getName()+" , dang o: " + a.getAddr()+", tuoi: "+a.getAge());
    }

}
```

Hình 4. 6. Chương trình minh họa tính kế thừa từ 2 lớp

Kết quả thực thi cụ thể hình bên dưới.



Hình 4. 7. Kết quả minh họa

4.2. Thiết kế kiến trúc

4.2.1. Vai trò kiến trúc phần mềm

4.2.1.1. Định nghĩa kiến trúc

Kiến trúc phần mềm *(Software Architecture)* chỉ một cấu trúc tổng thể của phần mềm, và qua đó cung cấp một sự tích hợp về mặt khái niệm của *một hệ thống*. Ở dạng đơn giản nhất, kiến trúc là một cấu trúc phân cấp của các thành phần chương trình, qua đó thể hiện sự tương tác giữa chúng với nhau, và các cấu trúc dữ liệu chúng sử dụng. Theo nghĩa rộng, kiến trúc biểu diễn các thành phần lớn, cốt lõi của hệ thống và mối quan hệ giữa chúng được nhìn theo nhiều quan điểm (cấu trúc, hành vi, dữ liệu,…).

4.2.1.2. Vai trò và tầm quan trọng của kiến trúc

Kiến trúc có một vai trò quan trọng trong phát triển phần mềm. Bass và đồng nghiệp của ông đã chỉ ra ba vai trò của kiến trúc phần mềm.

- Công cụ giao tiếp giữa những người có liên quan.
- Để phân tích hệ thống: Khi đưa kiến trúc hệ thống ở giai đoạn đầu, nhà phân tích có thể tiến hành những quyết định thiết kế có tác động sâu sắc đến hệ thống sau này như tính hoàn thiện, độ ổn định, tính bảo trì,…
- Sử dụng lại ở quy mô lớn: Kiến trúc thu được có thể dùng cho các hệ thống có yêu cầu tương tự, và vì vậy hỗ trợ việc sử dụng lại với quy mô lớn.

Kiến trúc không phải thành phần hoạt động, nhưng có tác động sâu rộng đến toàn bộ tiến trình phát triển phần mềm. Nó là một mô tả về phần mềm cho phép các kỹ sư phần mềm thực hiện những công việc cụ thể.

- Tăng cường hiểu biết về hệ thống cần xây dựng: Mô hình kiến trúc mô tả hệ thống theo nhiều khía cạnh khác nhau, giúp các nhà phát triển có thể hình dung hệ thống cần xây dựng một cách đầy đủ và chính xác
- Phân tích hiệu quả: Với một tập các yêu cầu, có thể nhiều phương án kiến trúc khác nhau. Phân tích giúp nhà thiết kế lựa chọn một kiến trúc hiệu quả nhất trong điều kiện có thể.
- Xem xét, sửa đổi kiến trúc từ sớm, giảm rủi ro: Ở giai đoạn đầu khi sự thay đổi thiết kế còn dễ dàng, ít tốn kém, do đó việc phát hiện và sửa đổi những sai sót sẽ làm giảm thiểu được chi phí hay rủi ro có thể do kiến trúc gây ra.

4.2.2. Các mô hình kiến trúc

Kiến trúc phần mềm được mô tả ở những dạng các mô hình khác nhau. Các mô hình kiến trúc được phát triển thường có thể nhìn nhận dưới một số mặt như sau.

- Mô hình kiến trúc tĩnh: Đưa ra các hệ con hay các thành phần được phát triển như một đơn vị độc lập.
- Mô hình tiến trình động: Đưa ra hệ thống được tổ chức thành các tiến trình vận hành.
- Mô hình giao diện: Xác định các giao diện đưa ra các dịch vụ mà mỗi hệ con cung cấp thông qua giao diện của chúng.
- Mô hình liên kết: Chỉ ra mối liên kết giữa các hệ con hay giữa các thành phần.

4.2.2.1. Mô hình kho dữ liệu (*Repository Model*)

- Các phân hệ cần trao đổi thông tin. Việc này có thể tiến hành theo hai cách *mọi dữ liệu được lưu trữ tại một CSDL trung tâm có thể được truy cập bởi mọi phân hệ*. Mô hình này được gọi là mô hình kho dữ liệu.
- Mỗi phân hệ duy trì một CSDL riêng của mình. Dữ liệu được trao đổi giữa các phân hệ bằng cách chuyển đổi các thông báo.

Mô hình này phù hợp cho các ứng dụng khi dữ liệu được tạo bởi một phân hệ và được sử dụng bởi các phân hệ khác.

Đây là cách hữu hiệu để chia sẻ một số lượng lớn dữ liệu, mà không cần chuyển đổi dữ liệu tường minh từ một phân hệ này tới các phân hệ khác. Phân hệ phải chấp nhận mô hình này nếu muốn tham gia hệ thống. Sẽ rất khó tích hợp một phân hệ mới nếu nó không phù hợp với tiêu chuẩn của kho dữ liệu. Phân hệ tạo dữ liệu không cần liên quan đến việc dữ liệu được phân hệ khác sử dụng như thế nào. Việc phát triển mô hình sẽ khó khăn khi một số lượng lớn dữ liệu đã có theo tiêu chuẩn cũ. Việc chuyển đổi dữ liệu sẽ rất tốn kém. Các hoạt động như lưu trữ, bảo mật, điều khiển truy nhập và khôi phục được tập trung hóa. Tuy nhiên, các phân hệ có thể có những yêu cầu khác nhau về mức độ bảo mật, khôi phục và chiến lược lưu trữ. Mô hình này bắt buộc các phân hệ phải chấp nhận một chính sách chung. Mọi việc sẽ đơn giản nếu phân hệ mới cần tích hợp tương thích với dữ liệu cũ. Tuy nhiên sẽ khó khăn nếu phân phối dữ liệu đến nhiều máy khác nhau. Việc này sẽ phát sinh khả năng dư thừa dữ liệu, không toàn vẹn.

4.2.2.2. Mô hình khách - Phục vụ (Client - Server Model)

Mô hình khách - phục vụ là một mô hình hệ thống phân tán biểu diễn việc phân tán các dữ liệu và xử lý trên nhiều máy tính khác nhau. Các thành phần chính.

- Một tập các server độc lập phục vụ cho các phân hệ.
- Một tập các khách hàng yêu cầu các dịch vụ. Chúng có thể là các phân hệ. Chúng cũng có thể là các thể hiện khác nhau của cùng một chương trình.
- Một mạng cho phép các khách hàng có thể truy nhập được các dịch vụ.

Khách hàng phải biết được định danh của các dịch vụ, còn các dịch vụ không cần biết các định danh của khách hàng.

Ưu điểm quan trọng nhất của mô hình này là sự phân tán rất rõ. Mô hình này dễ dàng thêm một server, và tích hợp dần dần khi có nhu cầu mà không ảnh hưởng đến các thành phần cũ. Sự thiếu vắng mô hình chia sẻ dữ liệu ở đây có nghĩa là sẽ khó dự đoán được các vấn đề khi tích hợp dữ liệu vào hệ thống cũ. Mỗi server phải có trách nhiệm với bản thân mình về lưu trữ, khôi phục,… Không có một trung tâm nếu các

khách hàng phải tự biết và tìm server. Đối với mạng LAN thì không có vấn đề nhưng đối với các mạng lớn như WAN, Internet thì có thể nảy sinh vấn đề.

4.2.3. Phân tích Module

Sau khi mô hình cấu trúc đã được thiết kế, mức tiếp theo của việc phân tích là phân rã các phân hệ thành các module. Các mô hình có thể được áp dụng cho mức này, tuy nhiên các cấu thành trong các module thường nhỏ hơn các phân hệ. Hai mô hình thường được sử dụng.

- Mô hình hướng đối tượng. Hệ thống được phân chia thành các đối tượng giao tiếp với nhau.
- Mô hình luồng dữ liệu. Hệ thống được phân chia thành các module chức năng. Chúng nhận các dữ liệu chuyển hóa chúng rồi lại đưa ra kết quả.

Trong mô hình hướng đối tượng, các module là các đối tượng với các trạng thái riêng và các hoạt động trên các trạng thái này. Trong mô hình luồng dữ liệu, module là các bộ biến đổi chức năng.

4.2.3.1. Mô hình đối tượng

Một mô hình hướng đối tượng bao gồm các đối tượng được định nghĩa giao diện tốt. Các đối tượng gọi là các dịch vụ từ các đối tượng khác. Một phân tích hướng đối tượng liên quan các lớp đối tượng, các thuộc tính và các hoạt động của chúng. Ưu điểm của hướng đối tượng được biết rõ. Tính bao đóng cho phép dấu các thực hiện của các đối tượng và cho phép dùng lại mã. Tuy nhiên, nhược điểm là để sử dụng các dịch vụ, các đối tượng phải gọi tường minh các tên và giao diện của các đối tượng khác. Sự thay đổi giao diện sẽ làm ảnh hưởng đến các đối tượng khác.

4.2.3.2. Mô hình luồng dữ liệu

Trong mô hình luồng dữ liệu, các bộ biến đổi xử lý dữ liệu đầu vào và tạo dữ liệu ra. Dữ liệu được chảy theo luồng từ bộ biến đổi này sang bộ khác tuần tự. Mỗi bước của quy trình giống như một phép biến đổi. Ưu điểm của kiến trúc này.

- Hỗ trợ việc sử dụng lại các biển đổi.
- Phù hợp với suy nghĩ của mọi người trên cơ sở quan niệm về dữ liệu được xử lý theo luồng có đầu vào và đầu ra.
- Thêm các xử lý khác vào hệ thống đơn giản.
- Dễ thực hiện xử lý song song hoặc tuần tự.

Nhược điểm của mô hình này.

- Cần phải có một định dạng chung cho các dữ liệu để có thể xử lý bởi mọi bộ biến đổi.
- Các hệ thống tương tác khó được viết theo mô hình luồng dữ liệu.

4.3. Thiết kế giao diện người - máy

4.3.1. Vai trò và tiến hóa giao diện người - máy

Ngày nay, khi có hàng triệu người trực tiếp sử dụng máy tính, thì giao diện người - máy chính là phương tiện thiết lập lên một cơ chế, qua đó người dùng có thể giao tiếp và sử dụng hệ thống máy tính. Một giao diện khó sử dụng gây nhiều khó khăn và dễ tạo ra sai lầm của người sử dụng. Trong trường hợp bình thường, mức lỗi gặp phải có thể rất cao, làm người dùng không muốn sử dụng hệ thống nữa. Trong trường hợp tồi tệ nhất, nó có thể làm cho hệ thống trở nên không khả thi. Chính vì vậy, giao diện của hệ thống tương tác thường được xem là *tiêu chuẩn để đánh giá hệ thống tồi hay tốt*. Một hệ thống có giao diện tốt là một hệ thống thành công.

Giao diện tương tác người - máy đã tiến hóa qua nhiều thế hệ.

- *Thế hệ thứ nhất* giao diện chỉ lệnh và hỏi.
- *Thế hệ thứ hai* giao diện thực đơn (menu) đơn giản.
- *Thế hệ thứ ba* giao diện đồ họa. Ngày nay giao diện người dùng đồ họa GUI đã quen thuộc với nhiều độc giả.

Ưu điểm của giao diện người dùng đồ họa dễ học, dễ sử dụng, có nhiều cửa sổ cho phép hiển thị đồng thời nhiều kiểu thông tin khác nhau. Người dùng có thể thực hiện đồng thời nhiều công việc, có thể chuyển cảnh mà không mất mối liên hệ trực quan với công việc khác. Việc dùng biểu tượng đồ họa, thực đơn kéo thả, nút và các kỹ thuật khác làm giảm số lượng thao tác, tăng hiệu quả tương tác cho người dùng.

4.3.2. Các nguyên tắc thiết kế giao diện người - máy

Giao diện người dùng được xem là yếu tố quyết định sự thành công của một hệ thống. Khi nhân tố con người được tính tới thì đối thoại sẽ trôi chảy, và nhịp điệu giao tiếp sẽ được thiết lập giữa người dùng và hệ thống. Vì thế, *thiết kế giao diện người sử dụng phải lấy con người làm trung tâm*. Người dùng tiềm năng nên được tham gia như thành viên chính thức của đội thiết kế giao diện. Bảng liệt kê một số nguyên tắc chính cho thiết kế giao diện.

Nguyên tắc	**Mô tả**
Tính thân thiện với người sử dụng (*user familiarity*)	Giao diện nên sử dụng các thuật ngữ và các khái niệm gần gũi được rút ra từ kinh nghiệm của những người sử dụng hệ thống.
Tính nhất quán (*consistency*)	Giao diện nên nhất quán ở những chỗ so sánh nên được kích hoạt theo cùng một cách.
Làm ngạc nhiên nhỏ nhất (*minimal surprise*)	Các hoạt động của hệ thống không được làm người sử dụng phải ngạc nhiên.
Có khả năng phục hồi (*recoverability*)	Giao diện nên có các cơ chế cho phép người dùng phục hồi lại trạng thái trước đó mỗi khi gặp lỗi.
Hướng dẫn người sử dụng (*user guidance*)	Giao diện cần cung cấp các thông tin phản hồi có ý nghĩa và cung cấp các tiện ích trợ giúp người sử dụng theo từng ngữ cảnh nhạy cảm.
Đa dạng người dùng (*user diversity*)	Giao diện cần cung cấp các tiện ích tương tác thích hợp với nhiều loại người dùng khác nhau.

Bảng 4. 1. Các nguyên tắc thiết kế giao diện người sử dụng

Tiến trình thiết kế giao diện gồm một số hoạt động sau.

- Bắt đầu tạo ra các mô hình khác nhau về chức năng hệ thống.
- Phác họa các nhiệm vụ hướng con người và máy để đạt đến chức năng hệ thống.
- Xem xét các giải pháp thiết kế được áp dụng cho mọi thiết kế giao diện.
- Sử dụng các công cụ làm bản mẫu.
- Cài đặt cho mô hình thiết kế và đánh giá kết quả về chất lượng.

4.4. Các phương pháp thiết kế

Mỗi phương pháp phân tích đều tương ứng với một phương pháp thiết kế bao gồm một tập các mô tả thiết kế, và các ký pháp cho phép các kỹ sư phần mềm tạo ra các mô hình thiết kế nhất quán.

4.4.1. Phương pháp Booch

Phương pháp Booch bao gồm cả hai quy trình phát triển vi mô và vĩ mô. Mức vi mô xác định tập các nhiệm vụ thiết kế được áp dụng lại cho các bước trong quy trình vĩ mô. Do đó, cách tiếp cận tiến hóa được duy trì. Một phác thảo của quy trình phát triển vi mô do Booch đề nghị.

- Lập kế hoạch kiến trúc
  - Nhóm các đối tượng tương tự nhau vào các phần kiến trúc riêng rẽ.
  - Phân lớp đối tượng theo mức trừu tượng.
  - Xác định các kịch bản thích hợp.
  - Tạo ra các nguyên mẫu thiết kế.
  - Hợp lệ các nguyên mẫu thiết kế bằng cách áp dụng cho các kịch bản.
- Chiến lược thiết kế
  - Xác định chính sách phụ thuộc lĩnh vực (ví dụ các luật quản lý việc sử dụng các phương thức và các thuộc tính)
  - Xác định chính sách mô tả lĩnh vực cho quản lý bộ nhớ, quản lý lỗi và các chức năng hạ tầng khác.
  - Phát triển kịch bản mô tả ngữ nghĩa của chính sách.
  - Tạo ra các nguyên mẫu cho mỗi chính sách.
  - Phối hợp và tinh chế các nguyên mẫu.
  - Xem xét lại các chính sách để đảm bảo rằng nó bao phủ hết các khung nhìn kiến trúc.
- Lập kế hoạch phát hành
  - Tổ chức các kịch bản được phát triển trong phân tích theo độ ưu tiên.
  - Cấp phát các kiến trúc khả thi cho các kịch bản.
  - Thiết kế và xây dựng mỗi kiến trúc khả thi tăng dần.
  - Điều chỉnh mục tiêu và lịch trình của các khả thi theo yêu cầu.

4.4.2. Phương pháp Coad và Yourdon

Phương pháp này được phát triển bằng cách nghiên cứu cách thức thiết kế hiệu quả hướng đối tượng. Cách tiếp cận thiết kế này không chỉ cho ứng dụng mà cho cả các hạ tầng của ứng dụng. Một phác thảo của quy trình phát triển do Coad và Yourdon đề nghị.

- Thành phần miền vấn đề
  - Nhóm các lớp mô tả miền.
  - Thiết kế một cấu trúc cây lớp phù hợp cho ứng dụng.
  - Đơn giản thừa kế.
  - Tinh chế thiết kế để cải thiện hiệu năng.
  - Phát triển giao diện với các thành phần quản lý dữ liệu.
  - Xem xét lại thiết kế.
- Thành phần tương tác với con người
  - Xác định các tác nhân con người.
  - Xác định các kịch bản công việc.
  - Thiết kế cây lệnh người dùng.

- Tinh chế dãy các câu lệnh người dùng.
- Tích hợp các lớp GUI thích hợp.

o Thành phần quản lý công việc

- Xác định kiểu công việc.
- Thiết lập mức ưu tiên.
- Xác định công việc để phục vụ như người điều phối các công việc khác.
- Thiết kế đối tượng phù hợp cho mỗi công việc.

o Thành phần quản lý dữ liệu

- Thiết kế cấu trúc dữ liệu và các Layout.
- Thiết kế các dịch vụ yêu cầu để quản lý cấu trúc dữ liệu.
- Xác định các công cụ có thể trợ giúp trong thực hiện quản lý dữ liệu.
- Xác định các lớp tích hợp và các cây lớp.

4.4.3. Phương pháp Jacobson

Phương pháp tập trung nhấn mạnh vào tính theo vết mô hình phân tích OOSE (*Object - Oriented Software Engineering*). Một phác thảo của quy trình phát triển do Jacobson đề nghị.

o Xem xét sự thích nghi để các mô hình phân tích lý tưởng phù hợp với môi trường thế giới thực.

o Tạo ra các khối như các đối tượng thiết kế cơ bản.

- Xác định một khối để thực hiện liên quan đến đối tượng phân tích.
- Xác định khối giao diện, khối thực thể và khối điều khiển.
- Mô tả cách thức các khối liên lạc khi thực hiện.
- Xác định các kích thước được chuyển giữa các khối và thứ tự liên lạc.

o Tạo ra một biểu đồ tương tác cho biết cách các kích thước được chuyển giữa các khối.

o Tổ chức các khối trong các hệ thống con.

o Xem xét lại công việc thiết kế.

4.5. Câu hỏi và bài tập

Câu 1. Tầm quan trọng của thiết kế phần mềm? Các giai đoạn thiết kế cần trải qua?

Câu 2. Các bước thiết kế kiến trúc một hệ thống? Các loại kiến trúc thường gặp?

Câu 3. Các bước thiết kế hướng cấu trúc? Nêu ưu và nhược điểm của nó?

Câu 4. Các phương pháp thiết kế phần mềm?

Câu 5. Mô tả các hoạt động chính trong quy trình thiết kế phần mềm và kết quả đầu ra của các hoạt động?

Câu 6. Kiến trúc phần mềm (Sofware Architecture) là gì?

Câu 7. Khi mô tả 1 hệ thống phần mềm, tại sao phải thiết kế kiến trúc hệ thống trước khi hoàn tất đặc tả yêu cầu?

CHƯƠNG 5. LẬP TRÌNH

5.1. Giới thiệu

Khi làm việc trên các ứng dụng ta không có sự lựa chọn về ngôn ngữ. Nhưng nếu ban đầu chọn sai ngôn ngữ thì chúng ta phải liên tục sửa đổi yêu cầu để phù hợp với những giới hạn của ngôn ngữ. Trong phần này, chúng ta sẽ bàn về những đặc trưng của ngôn ngữ và cách lựa chọn ngôn ngữ lập trình dựa trên các tiêu chí ứng dụng.

Trước tiên, chúng ta định nghĩa các đặc trưng của ngôn ngữ. Ngôn ngữ lập trình là phương tiện để liên lạc giữa con người và máy tính. Tiến trình lập trình - sự liên lạc thông qua ngôn ngữ lập trình - là một hoạt động trí tuệ rất cao của con người (lập trình viên), và là một bước cốt lõi.

Cách nhìn công nghệ phần mềm về các đặc trưng của ngôn ngữ lập trình tập trung vào nhu cầu xác định dự án phát triển mỗi phần mềm. Mặc dù, người ta vẫn cần các yêu cầu riêng cho mã nguồn gốc, có thể thiết lập được một tập hợp tổng quát những đặc trưng công nghệ.

- Dễ dịch thiết kế sang chương trình.
- Từ khóa gần với ngôn ngữ tự nhiên.
- Khả chuyển chương trình gốc.
- Có sẵn công cụ phát triển.
- Dễ bảo trì.

Bước lập trình bắt đầu sau khi thiết kế đã được xác định, xét duyệt và sửa đổi nếu cần. Về lý thuyết, việc sinh mã chương trình gốc từ một đặc tả chi tiết nên là một *ánh xạ trực tiếp*. Dễ dịch thiết kế sang chương trình là đặc trưng về việc một ngôn ngữ lập trình phản xạ gần gũi với các biểu diễn thiết kế. Một ngôn ngữ cài đặt trực tiếp cho các kết cấu có cấu trúc, các cấu trúc dữ liệu phức tạp, vào/ra đặc biệt, khả năng thao tác bit, và các kết cấu hướng đối tượng sẽ làm cho việc dịch từ thiết kế sang chương trình gốc dễ dàng.

Mặc dù những tiến bộ nhanh chóng trong tốc độ xử lý, và mật độ nhớ đã bắt đầu làm giảm nhẹ nhu cầu chương trình siêu hiệu quả, nhiều ứng dụng vẫn còn đòi hỏi các chương trình chạy nhanh, gọn. Các ngôn ngữ với trình biên dịch tối ưu có thể là hấp dẫn nếu hiệu năng phần mềm là yêu cầu cốt lõi.

Tính khả chuyển chương trình gốc là một đặc trưng quan trọng khác của ngôn ngữ lập trình. Nó có thể được hiểu theo ba cách khác nhau.

- Chương trình gốc có thể chuyển từ bộ xử lý này sang bộ xử lý khác, và trình biên dịch này sang trình biên dịch kia với rất ít hoặc không phải sửa đổi gì.
- Chương trình gốc vẫn không thay đổi ngay cả khi môi trường của nó thay đổi.
- Chương trình gốc có thể được tích hợp vào trong các phần mềm khác nhau với ít hay không cần thay đổi gì vì các đặc trưng của ngôn ngữ lập trình.

Trong số ba cách hiểu về cách khả chuyển này thì cách thứ nhất là thông dụng nhất. Tính sẵn có của công cụ phát triển có thể làm giảm bớt thời gian cần để sinh ra chương trình gốc, và có thể cải thiện chất lượng của chương trình. Nhiều ngôn ngữ lập trình có thể cần đến một loạt các công cụ kể cả trình biên dịch gỡ lỗi, trợ giúp định dạng chương trình gốc, các tiện nghi soạn thảo có sẵn, các công cụ kiểm soát chương trình gốc, thư viện chương trình con mở rộng trong nhiều lĩnh vực ứng dụng, các trình duyệt, trình biên dịch chéo cho phát triển bộ vi xử lý.

Tính dễ bảo trì của chương trình gốc có tầm quan trọng chủ chốt cho tất cả các nổ lực phát triển phần mềm. Việc bảo trì không thể được tiến hành khi chừng nào người ta còn chưa hiểu được phần mềm. Các yếu tố của cấu hình phần mềm (như tài liệu thiết kế) đưa ra một nền tảng cho việc hiểu biết, cuối cùng thì chương trình gốc vẫn phải được đọc và sửa đổi theo những thay đổi trong thiết kế.

Tính dễ dịch thiết kế sang chương trình là một yếu tố quan trọng để dễ bảo trì chương trình gốc. Bên cạnh đó, các đặc trưng tự làm tư liệu của ngôn ngữ có ảnh hưởng mạnh đến tính dễ bảo trì. Một đặc trưng khác liên quan là các từ khóa gần với ngôn ngữ tự nhiên. Điều này cũng làm cho chương trình dễ hiểu, dễ bảo trì, cũng như dễ dịch thiết kế sang chương trình. Tuy nhiên, mặt trái của đặc trưng này là làm cho chương trình gốc trở nên phức tạp, đôi khi không thích hợp với môi trường lập trình chuyên nghiệp.

5.2. Phương pháp lập trình

5.2.1. Trừu tượng hóa trong lập trình

Trừu tượng hóa là một quá trình tư duy tự nhiên của con người khi tìm cách nhận thức một sự vật, vấn đề. Có thể diễn giải một cách đơn giản "*Trừu tượng hóa*" là sự hình thành khái niệm về sự vật hay hiện tượng bằng cách tập trung vào các đặc trưng cần thiết, và loại bỏ các chi tiết cụ thể không cần thiết.

Trong việc thiết kế chương trình, chúng ta phải ánh xạ một bài toán thực thành các thành phần của chương trình như các cấu trúc dữ liệu, cấu trúc điều khiển,… Các đối tượng trong bài toán thực tế có thể chứa rất nhiều các thuộc tính khác nhau, nhưng khi ánh xạ vào thành phần trong chương trình thì chúng ta chỉ mô tả các thuộc tính cần thiết mà chúng ta quan tâm. Không những thế, các thuộc tính này còn bị rời rạc hóa và mô tả trong một không gian hữu hạn. Đó là quá trình trừu tượng trong tiến trình phát triển phần mềm. Các kết quả của quá trình trừu tượng hóa này gọi là các *mô hình*. Với phần mềm, có thể có các mô hình như mô hình dữ liệu, mô hình giao diện,…

Chẳng hạn, để giải quyết một bài toán quản lý sinh viên của trường đại học, chúng ta có thể trừu tượng hóa khái niệm sinh viên thực thành mô hình sinh viên, bao gồm các thuộc tính như mã sinh viên, họ tên, điểm trung bình. Với bài toán này chúng ta có thể bỏ qua các thuộc tính như chiều cao, cân năng,… Tuy nhiên với một bài toán quản lý sức khỏe cộng đồng thì các thuộc tính này lại có thể là các thuộc tính cơ bản.

Trừu tượng hóa là chìa khóa then chốt trong phát triển phần mềm. Nếu tiến hành trừu tượng hóa tốt sẽ đảm bảo phát triển được phần mềm có chất lượng và dễ bảo trì.

5.2.2. Lập trình thủ tục

Lập trình thủ tục (*Procedural Programming*) là phương pháp lập trình tiên tiến bao gồm hai khái niệm chính.

- Sử dụng các thủ tục. Chia chương trình lớn thành các đoạn chương trình con (thủ tục) thực hiện các công việc xác định. Các thủ tục có thể được gọi nhiều lần, gọi lẫn nhau và gọi chính nó (đệ quy). Đây chính là ý tưởng chia để trị (*Divide anh Conquer*). Nó góp phần giảm độ phức tạp và tăng tính hiểu của chương trình.
- Lập trình có cấu trúc *(Structured Programming)*. Sử dụng các câu lệnh có cấu trúc hoặc tạo ra các cấu trúc phức tạp từ các cấu trúc đơn giản hơn. Câu lệnh có cấu trúc là các câu lệnh có mục đích sử dụng rõ ràng, điểm bắt đầu và điểm kết thúc câu lệnh tường minh. Điều đó làm cho chương trình trở nên dễ hiểu và sẽ khiến cho việc lập trình dễ dàng hơn.

Việc chia để trị cũng như việc sử dụng các câu lệnh có cấu trúc khiến cho chương trình dễ hiểu, dễ tái sử dụng. Đồng thời, nó cũng góp phần loại bỏ khái niệm lập trình cổ điển lạm dụng câu lệnh GOTO.

5.2.3. Lập trình hướng đối tượng

Phương pháp lập trình hướng đối tượng (*Object - Oriented Programming*) xuất hiện vào thập kỷ 70 của thế kỷ 20, và nhanh chóng trở thành phương pháp chủ đạo trong phát triển các phần mềm lớn và chuyên nghiệp. Lập trình hướng đối tượng giúp phát triển các phần mềm có chức năng phức tạp hơn, có hiệu quả hơn và dễ bảo trì hơn.

Lập trình hướng đối tượng dựa trên ba đặc trưng cơ bản là đóng gói/che dấu thông tin, kế thừa và đa hình.

Đóng gói/che dấu thông tin *(Encapsulation/Information hiding)* là đặc trưng cơ bản nhất của lập trình hướng đối tượng. Đóng gói/che dấu thông tin bao gồm hai công việc liên quan mật thiết với nhau.

- Việc gắn kết dữ liệu và các phương thức xử lý đặc trưng với dữ liệu thành một thể thống nhất gọi là (*lớp*) đối tượng.
- Việc che dấu các cài đặt và cấu trúc dữ liệu cụ thể của đối tượng, đối tượng cung cấp các dịch vụ thông qua giao diện xác định.

Thực hiện tốt việc đóng gói/che dấu thông tin sẽ cho phép chúng ta thao tác với đối tượng mà không cần biết cách thức cài đặt cụ thể của nó, tức là chúng ta tách được phần bất biến (giao diện) khỏi phần khả biến (một cách cài đặt cụ thể) của đối tượng. Do việc sử dụng đối tượng chỉ cần biết thông tin tối thiểu về giao diện nên chúng ta dễ dàng tái sử dụng, đồng thời việc bảo trì là dễ dàng. Hơn nữa, do chỉ thay đổi được trạng thái của đối tượng thông qua các giao diện xác định nên có thể đảm bảo được các trạng thái của đối tượng là luôn đúng đắn.

Kế thừa *(Inheritace)* là cơ chế đặc biệt của lập trình hướng đối tượng cho phép xây dựng một lớp đối tượng mới kế thừa (*sử dụng lại*) lớp đối tượng đã có. Kế thừa thúc đẩy việc sử dụng lại trong bản thân một phần mềm cũng như giữa các phần mềm với nhau.

Đa hình *(Polymorphism)* là cơ chế cho phép nhìn nhận một đối tượng theo các góc nhìn khác nhau, đồng thời cho phép các đối tượng khác nhau giải nghĩa cùng một thông điệp theo cách thức khác nhau. Cùng với kế thừa, đa hình tạo nên sự mềm dẻo đặc biệt trong lập trình hướng đối tượng. Đa hình đồng thời là cơ chế nền tảng cho việc thực hiện các thiết kế mở và xây dựng các thiết kế mẫu.

5.3. Một số môi trường phát triển

5.3.1. Môi trường VISUAL STUDIO.NET

Visual studio mà gần đây là Visual Studio.NET là một tập các công cụ phát triển hoàn chỉnh để xây dựng các ứng dụng chạy trên nền Windows (sử dụng Windows Forms, một phần của .NET Framework), Web (sử dụng ASP.NET và XML Web services), các ứng dụng desktop, và các ứng dụng thiết bị thông minh như PDAs, mobile phones, Pocket PC,…(sử dụng .NET Compact Framework). Web ASP, XML Web services, Visual Basic.NET, Visual C++.NET, Visual C#.NET và Visual J#.NET tất cả cùng sử dụng một môi trường phát triển tích hợp (*IDE - Integrated Development Environment*), cho phép chúng chia sẻ các công cụ và tiện ích trong việc tạo ra các giải pháp với nhiều ngôn ngữ. Có thể nói, VS.NET là một môi trường tích hợp nhiều ngôn

ngữ lập trình, là sự mở rộng của các ngôn ngữ hiện hành với cú pháp tương đương, chạy trên nền .NET framework.

Visual J# là một công cụ phát triển mà những người lập trình đã quen với ngôn ngữ Java có thể sử dụng, để xây dựng các ứng dụng và dịch vụ trên nền .NET Framework. Nó tích hợp cú pháp của ngôn ngữ Java vào môi trường phát triển tích hợp Visual Studio.NET. Visual J# cũng hỗ trợ hầu hết các chức năng có trong Visual J++ 6.0. Tuy nhiên, Visual J# không phải là một công cụ phát triển chạy trên máy ảo Java, các ứng dụng viết bằng Visual J# chỉ có thể thực hiện trên .NET Framework.

Visual C# là một ngôn ngữ lập trình mới được phát triển trong môi trường VS.NET, thích hợp cho việc xây dựng các ứng dụng doanh nghiệp, với cú pháp đơn giản tương tự như Visual Basic. C# cũng chứa một số thành phần giống Java, bao gồm cú pháp, bộ thu gom rác và không gian tên phân cấp.

Thay đổi lớn nhất của VS.NET là đưa ra môi trường phát triển các mã chương trình quản lý (*managed code*), sử dụng .NET Framework. Các chương trình được phát triển sử dụng. .NET không được biên dịch sang ngôn ngữ máy (ví dụ như C++) mà thay vào đó chúng được biên dịch sang một dạng MSIL (*Microsoft Intermediate Language*) hoặc CLI (*Common Language Infrastructure*). Khi một ứng dụng MSIL được thực thi, nó được biên dịch sang ngôn ngữ máy thích hợp với platform mà nó đang chạy trên đó, do đó mã chương trình có thể được thực thi trên nhiều nền. Có thể chạy MSIL trên Linux, hoặc Mac OS X sử dụng các cài đặt non - Microsoft.NET như Mono và DotGNU.

Visual Studio .NET mang đến cho các nhà lập trình cũng như người phát triển một giao diện làm việc thân thiện, và sự kết dính các thành phần tốt hơn. Nó tùy biến hơn cho người dùng với các điều khiển, các thanh công cụ kéo thả, và các cửa sổ công cụ tự động ẩn đi khi không sử dụng tới để tiết kiệm không gian làm việc. Với việc tích hợp các điều khiển của người dùng vào thanh công cụ, nó mang lại cho người dùng khả năng thay đổi, bổ sung theo ý muốn các điều khiển phù hợp với ứng dụng của mình.

VS .NET cung cấp các mô hình lập trình phong phú (còn được gọi là các mô hình đối tượng tự động - Automation Object Models) hỗ trợ cho việc mở rộng và tự động hóa môi trường phát triển. Mỗi mô hình tập trung vào một lĩnh vực cụ thể của IDE như tool Windows, code editor, các ấn bản dành cho nhà phát triển phần mềm, các nhà kiểm thử phần mềm, các chuyên gia cơ sở dữ liệu,…

Bằng cách sử dụng Visual Paradigm Smart Development Environment (SDE), có thể chuyển các thiết kế hệ thống (sơ đồ UML) sang mã cài đặt chương trình (mã nguồn C#, C++ hoặc VB .NET,…). Ví dụ sinh mã C# từ sơ đồ lớp và ngược lại sinh sơ đồ lớp từ mã C#, cũng như tạo ra các báo cáo.

Visual Studio cung cấp các tiện ích cho phép các nhà lập trình viết các add - ins để mở rộng khả năng của nó. Những thành phần tích hợp này được "*gắn*" vào Visual Studio và cung cấp các tiện ích mà bản thân Visual Studio không có.

5.3.2. Môi trường lập trình Java

Java là ngôn ngữ lập trình được sử dụng khá phổ biến hiện nay. Ưu điểm chính của Java là khả năng chạy độc lập nền và tính hướng đối tượng hoàn toàn. Vì thế, độ an toàn cũng như khả năng sử dụng mã của Java là rất cao.

Để biên dịch và chạy chương trình Java, chúng ta cần cài đặt "bộ công cụ phát triển Java" (*Java Development Kit - JDK*). Các hệ điều hành khác nhau sẽ sử dụng các bản JDK tương ứng với nó.

Để hỗ trợ cho những người dùng mới tiếp cận với Java cũng như để giúp quản lý các dự án lớn, một số công ty đã phát triển các bộ môi trường phát triển tích hợp (*Integrated Development Environment - IDE*). Các bộ công cụ này nhiều mức độ khác nhau và thích hợp với từng công việc cụ thể. Các bộ soạn thảo đơn giản đòi hỏi ít tài nguyên như TextPad, EditPlus,… thường chỉ hỗ trợ các công việc đơn giản như đổi màu từ khóa, biên dịch hay chạy chương trình từ menu. Trong khi đó, một số môi trường mạnh hơn, đòi hỏi nhiều tài nguyên hơn như IntelliJ, Jbuilder, NetBean, Eclipse,… cho phép thực hiện các công việc phức tạp hơn như xây dựng giao diện đồ họa bằng kéo thả, quản lý các dự án nhiều file, quản lý phiên bản,…

Một trong số các IDE được nhiều lập trình viên ưa thích hiện nay là Eclipse. Đây là một IDE mã nguồn mở do công ty IBM phát triển. Eclipse cung cấp sẵn một số công cụ cơ bản như hỗ trợ biên dịch, gỡ lỗi, hoặc hỗ trợ quản lý, đóng gói các dự án lớn. Điểm thú vị nhất của Eclipse là môi trường này hoạt động như một Framework, chúng cho phép người dùng mở rộng, tích hợp các plug - in từ bên ngoài vào. Qua đó người sử dụng có thể dễ dàng xây dựng mới hoặc sử dụng lại các công cụ phần mềm của hãng thứ ba ngay trong môi trường Eclipse. Hiện nay, Eclipse đã có nhiều phiên bản hỗ trợ cho nhiều môi trường khác nhau như Windows, Linux,… Toàn bộ chương trình, mã nguồn cũng như tài liệu trợ giúp về Eclipse có thể được tải miễn phí từ địa chỉ httlp://www.eclipse.org.

5.4. Phong cách lập trình

Phong cách lập trình bao hàm một triết lý về lập trình nhấn mạnh tới tính dễ hiểu của chương trình nguồn. Các yếu tố của phong cách bao gồm tài liệu bên trong chương trình, phương pháp khai báo dữ liệu, cách xây dựng câu lệnh và các kỹ thuật vào/ra.

5.4.1. Tài liệu chương trình

Tài liệu bên trong của chương trình gốc bắt đầu với việc chọn các tên gọi định danh (*biến và nhãn*), tiếp tục với vị trí và thành phần của việc chú thích, kết luận với cách tổ chức trực quan của chương trình.

Việc lựa chọn các tên gọi định danh là điều chủ chốt cho việc hiểu chương trình. Những ngôn ngữ giới hạn độ dài tên biến hay nhãn làm các tên mang nghĩa mơ hồ. Cho dù một chương trình nhỏ thì một tên gọi có nghĩa cũng làm tăng tính dễ hiểu. Theo ngôn từ của mô hình cú pháp/ ngữ nghĩa tên có ý nghĩa làm "*đơn giản hóa việc chuyển đổi từ cú pháp chương trình sang cấu trúc ngữ nghĩa bên trong*".

Một điều rõ ràng là "*phần mềm phải chứa tài liệu bên trong*". Lời chú thích cung cấp cho người phát triển một ý nghĩa truyền thông với các độc giả khác về chương trình gốc. Lời chú thích có thể cung cấp một hướng dẫn rõ rệt để hiểu trong pha cuối cùng của công nghệ phần mềm - bảo trì.

Có nhiều hướng dẫn đã được đề nghị cho việc viết lời chú thích. Các chú thích mở đầu và chú thích chức năng là hai phạm trù đòi hỏi cách tiếp cận có hơi khác.

Lời chú thích mở đầu nên xuất hiện ở ngay đầu của mọi module. Định dạng cho lời chú thích cụ thể như.

- Một phát biểu về mục đích chỉ rõ chức năng module.
- Mô tả giao diện.

- Mô tả mẫu cách gọi.
- Mô tả về dữ liệu.
- Danh sách tất cả các module thuộc cấp.

o Thảo luận về dữ liệu thích hợp (như các biến quan trọng và những hạn chế, giới hạn về cách dùng chúng) và các thông tin quan trọng khác.

o Lịch sử phát triển bao gồm.

- Tên tác giả.
- Tên người xét duyệt và ngày tháng.
- Ngày tháng sửa đổi và mô tả sửa đổi.

Các chú thích chức năng được nhúng vào bên trong thân của chương trình gốc và được dùng để mô tả cho các khối chương trình.

Việc viết tài liệu bên trong chương trình, ngày càng được chú trọng với sự xuất hiện các công cụ soạn thảo mã nguồn mạnh, hỗ trợ cho hiển thị chú thích theo các khuôn dạng thích hợp, và hỗ trợ sinh tự động các tài liệu ngoài từ chú thích. Các tài liệu mô tả ngoài sinh tự động theo kiểu này thường được tạo ra theo định dạng HTML và có các liên kết trỏ thẳng đến các thành phần tương ứng của chương trình.

5.4.2. Khai báo dữ liệu

Độ phức tạp của cấu trúc dữ liệu phụ thuộc vào thiết kế dữ liệu. Vấn đề này được thiết lập trong pha thiết kế. Tuy nhiên, người lập trình với việc tuân theo những chỉ dẫn đơn giản vẫn có thể cài đặt các cấu trúc dữ liệu dễ hiểu hơn, và thuận tiện hơn trong bảo trì. Dưới đây là một số hướng dẫn trong việc khai báo dữ liệu.

o Cấu trúc dữ liệu nên được chú giải đầy đủ về cấu trúc và chức năng, và các đặc thù về cách sử dụng. Đặc biệt là đối với các cấu trúc phức tạp như danh sách móc nối trong các ngôn ngữ như C. Tương tự như vậy, cần phải chú giải về mục đích sử dụng đối với các biến quan trọng, đặc biệt là các biến tổng thể.

o Thứ tự khai báo dữ liệu nên được chuẩn hóa cho dù ngôn ngữ lập trình không có yêu cầu bắt buộc nào về điều đó. Điều đó có nghĩa là các biến phụ thuộc cùng kiểu hoặc cùng mục đích sử dụng nên được xếp cạnh nhau.

o Cần thống nhất quy cách đặt tên cho các biến. Các tên biến ngoài việc thể hiện ý nghĩa thực hiện chức năng còn nên mang thông tin về kiểu của chúng. Nên thống nhất tên cho các biến cục bộ và các biến sử dụng vào các mục đích đặc biệt như chỉ mục trong vòng lặp.

5.4.3. Xây dựng câu lệnh

Thiết kế chi tiết các luồng logic của một chức năng phần mềm là công đoạn của pha thiết kế. Tuy nhiên việc xây dựng từng câu lệnh cụ thể để thể hiện thiết kế này là công việc quan trọng của pha lập trình. Để tránh lỗi và nâng cao khả năng bảo trì, việc xây dựng câu lệnh nên tuân thủ một quy tắc quan trọng *mỗi câu lệnh nên đơn giản và trực tiếp*.

Nhiều ngôn ngữ lập trình cho phép viết nhiều câu lệnh trên một dòng hoặc cho phép viết các câu lệnh phức tạp. Khía cạnh tiết kiệm không gian của tính năng này khó mà biện minh cho tính khó đọc nảy sinh theo nó. Cấu trúc của khối lệnh cũng như các phép toán điều kiện sẽ dễ dàng bị che lấp bởi cách lập trình như vậy.

Phong cách lập trình có cấu trúc được thể hiện thông qua một giải pháp đơn giản *xây dựng các câu lệnh đơn và tụt lề theo các cấp độ ứng với từng khối chức năng*.

Các câu lệnh có thể được đơn giản hóa bởi bằng các kỹ thuât.

o Tránh dùng các phép kiểm tra phức tạp. Phép kiểm tra điều kiện phức tạp sẽ làm tăng khả năng nhầm lẫn khi viết chương trình.

o Khử bỏ các phép kiểm tra điều kiện phủ định. Nếu có thể nên thay các điều kiện phủ định bằng điều kiện khẳng định, con người thường tư duy về điều kiện khẳng định dễ hơn nhiều so với chiều ngược lại.

o Tránh lồng nhau nhiều giữa các điều kiện hay chu trình. Việc các điều kiện và chu trình lồng nhau sẽ làm tăng mức độ phức tạp của chương trình, dẫn đến đoạn chương trình khó hiểu, khó bảo trì.

o Dùng dấu ngoặc để làm sáng tỏ các biểu thức logic hay số học. Nên đơn giản hóa các biểu thức phức tạp bằng các biểu thức đơn giản, hoặc tối thiểu dùng các dấu ngoặc để làm tăng tính tường minh của biểu thức.

o Dùng dấu cách và/hoặc các ký hiệu dễ đọc để làm sáng tỏ nội dung câu lệnh. Nên dùng các dấu cách hoặc các tên biến dễ đọc để làm tăng tính dễ đọc của câu lệnh.

Để hướng đến chương trình dễ hiểu luôn nên đặt ra câu hỏi: *Liệu người khác có thể hiểu được đoạn chương trình đó không?* Hay nói tóm lại, cố gắng giữ cho chương trình càng đơn giản càng tốt.

5.4.4. Phong cách vào/ra của các module

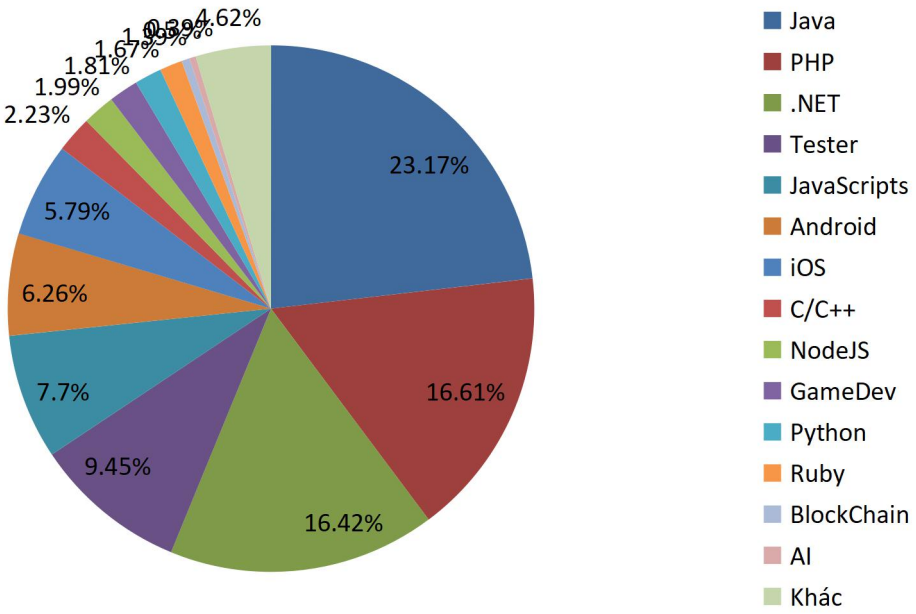
Vào ra của các chương trình tương tác với người sử dụng là một phạm trù phức tạp, và được bàn đến trong việc thiết kế giao diện - người máy. Ở đây, chúng ta quan tâm đến vào/ra của các module, hay nói cách khác sự tương tác giữa các module phần mềm.

Về cơ bản, thông tin cần trao đổi giữa các module được xác định ở pha thiết kế. Tuy nhiên ở pha lập trình, chúng ta vẫn có nhiều việc để làm nhằm đảm bảo cho chương trình dễ hiểu và hoạt động tin cậy hơn. Vào/ra giữa module nên tuân thủ theo một số hướng dẫn sau.

o Kiểm tra tính hợp lệ dữ liệu vào. Khi module được tích hợp vào một hệ thống phức tạp và trao đổi thông tin với các module do người khác thiết kế, chúng ta cần phải kiểm tra tính hợp lệ của thông tin vào, và đảm bảo module chỉ làm việc với các dữ liệu hợp lệ.

o Giữ cho định dạng dữ liệu vào đơn giản và thống nhất. Nên sử dụng các định dạng đơn giản trong việc vào/ra, điều này sẽ làm tăng tính dễ hiểu cũng như giảm chi phí truyền dữ liệu. Bên cạnh đó, chúng ta nên thống nhất các quy cách với dữ liệu vào/ra như kiểu, thứ tự của các khoản mục dữ liệu, quy cách của các thông tin về lỗi và ngoại lệ,…

o Giảm độ ghép nối. Một trong những phương pháp nâng cao chất lượng phần mềm là làm giảm độ ghép nối giữa các module. Ở pha lập trình, chúng ta có thể giảm độ ghép nối thông qua việc loại bỏ các trường dữ liệu không cần thiết.



Hình 5. 1. Nhu cầu tuyển dụng nhân lực phần mềm

5.5. Câu hỏi và bài tập

Câu 1. Thế nào là ngôn ngữ lập trình? Thế nào là lập trình?
Câu 2. Phân biệt các mức của ngôn ngữ lập trình?
Câu 3. Phân biệt giữa lỗi biên dịch (compiler) và lỗi thông dịch (interpreter)?
Câu 4. Viết chương trình đếm số chữ số của một số nguyên dương n?
Câu 5. Viết chương trình tính tổng 3 số tự nhiên nhập từ bàn phím?
Câu 6. Viết chương trình tính n!
Câu 7. Viết chương trình tìm ước số chung lớn nhất, bội số chung nhỏ nhất của hai số tự nhiên a và b?
Câu 8. Viết chương trình liệt kê tất cả các số nguyên tố nhỏ hơn n cho trước?
Câu 9. Viết chương trình liệt kê n số nguyên tố đầu tiên?
Câu 10. Viết chương trình giải phương trình bậc 2?
Câu 11. Viết chương trình phân tích số nguyên n thành các thừa số nguyên tố?